

Diffusion Model Paper Review

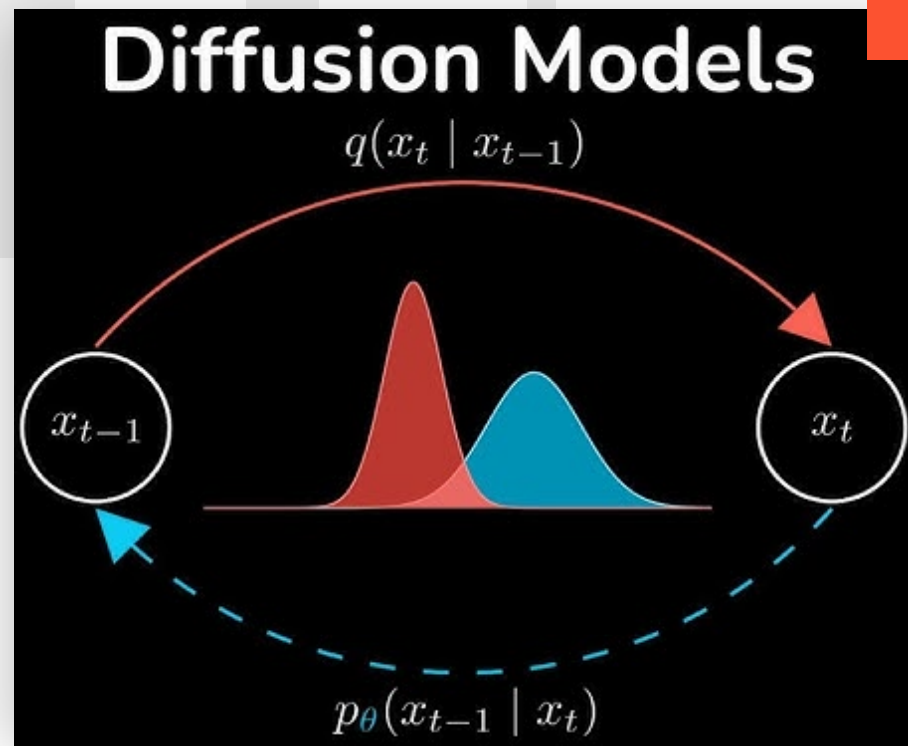
Deep Unsupervised Learning Using Nonequilibrium Thermodynamics - Sohl-Dickstein et al.

Denoising Diffusion Probabilistic Models - Jonathan Ho et al.

Denoising Diffusion Implicit Models - Jiaming Song et al.

Prompt-to-Prompt Image Editing with Cross Attention Contro - Amir Hertz et al.

SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis - Dustin Podell et al.



핵심 아이디어

작은 복원을 여러 번 합치면 큰 복원이 된다.

원본 데이터에 가우시안 노이즈를 조금씩 추가(Forward)해 완전히 섞인 가우시안 분포로 보낸 뒤, 그 과정을 거꾸로 되돌리는 확률적 변환(Reverse)을 신경망으로 학습하는 모델



제언

딥페이크 시대, AI 개발자의 윤리적 책임

딥페이크는 diffusion 모델에 기반한 기술로, 심각한 사회 문제로 떠오르고 있다. AI 개발자는 확산 모델의 원리와 한계를 이해하고, 예방·탐지·책임 배포까지 대처할 윤리적 책임과 실무적 의무가 있다.

■ 목차

01	GAN / VAE	04p
02	Diffusion Model	14p
03	DDPM / DDIM	18p
04	Stable Diffusion	24p
05	Code Implementation	32p



01



GAN / VAE

Before the diffusion model came out



GAN

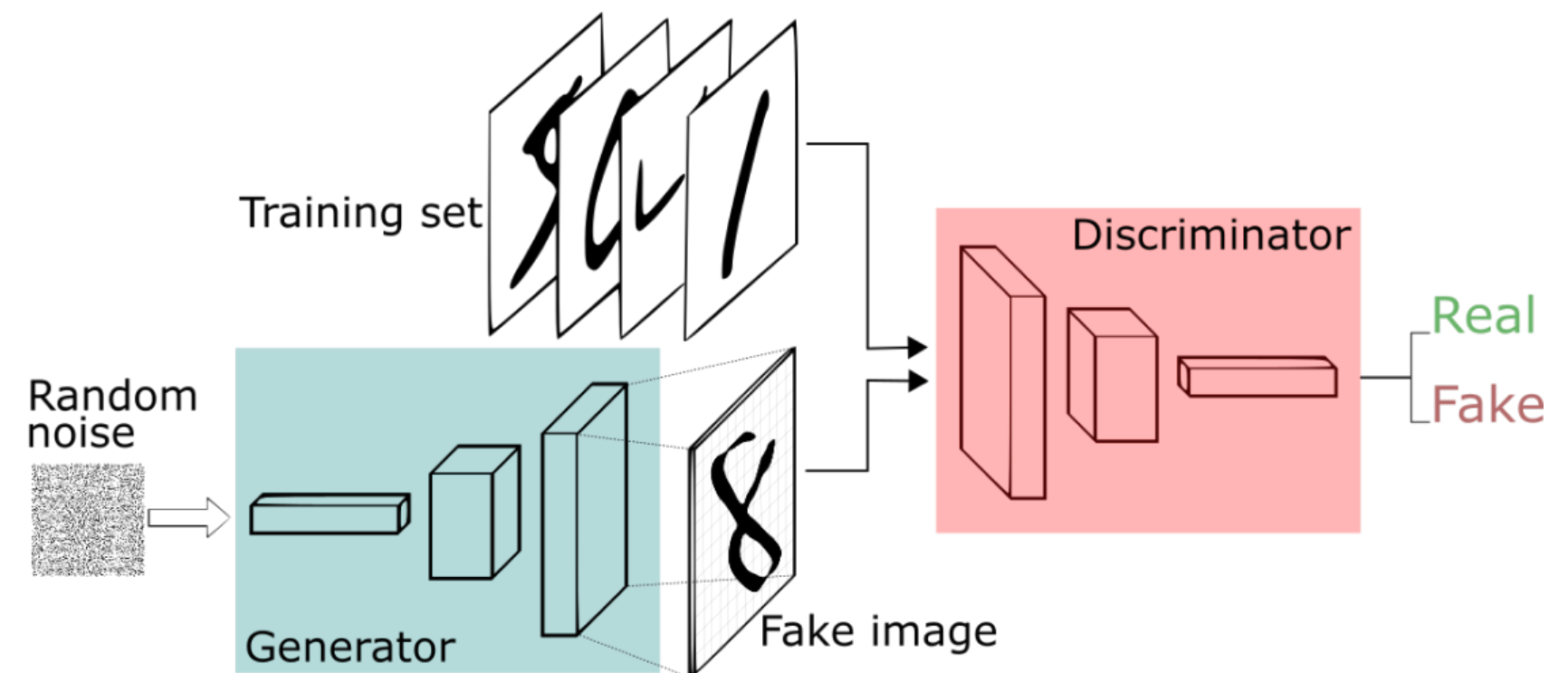
(Generative Adversarial Network/생성형 대립 신경망)

개요

실제 데이터의 Training Set과 Random Noise를 Generator를 통과시킴으로써 나온 Fake Image와 Discriminator에서 비교하여 Real과 Fake를 구분할 수 있는지 확인하면서 학습하는 모델

간단 설명)

Discriminator는 진짜/가짜를 잘 맞히려 하고,
Generator는 Discriminator를 속이려 한다
→ 적대적/대립적(Adversarial) 학습.



GAN

(Generative Adversarial Network/생성형 대립 신경망)

Generator(초기 모델)

1) 랜덤 숫자 조합 뽑기

Gaussian noise를 가진 Latent z (잠재 벡터 z) 를 추출

2) Fully-connected layer - reshape

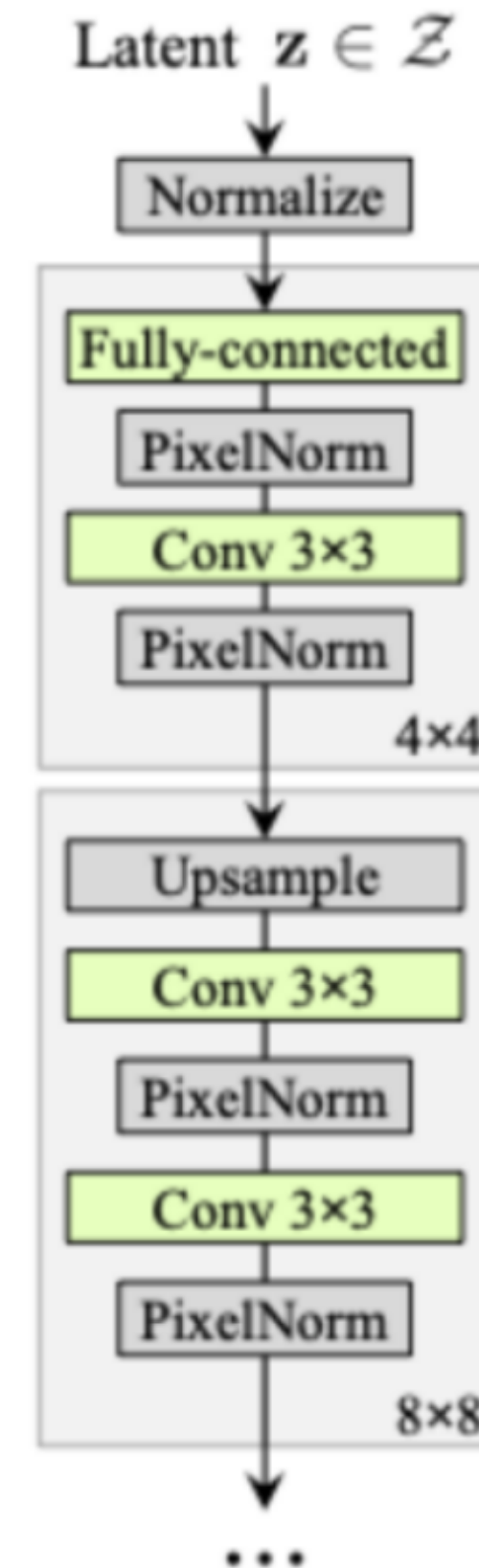
FC layer를 통과 시켜서 $C * 4 * 4$ 로 만들
reshape으로 $(C, 4, 4)$ 의 작은 이미지 형태로 변경

3) Upsample - (Conv $3*3$ - PixelNorm) 반복

이미지의 픽셀 수를 늘려가면서 반복

4) 출력

필터 수를 3(RGB)으로 맞추는 후 비교할 Training Data의
이미지 크기를 맞춤



GAN

(Generative Adversarial Network/생성형 대립 신경망)

Discriminator (초기 모델)

1) 원리

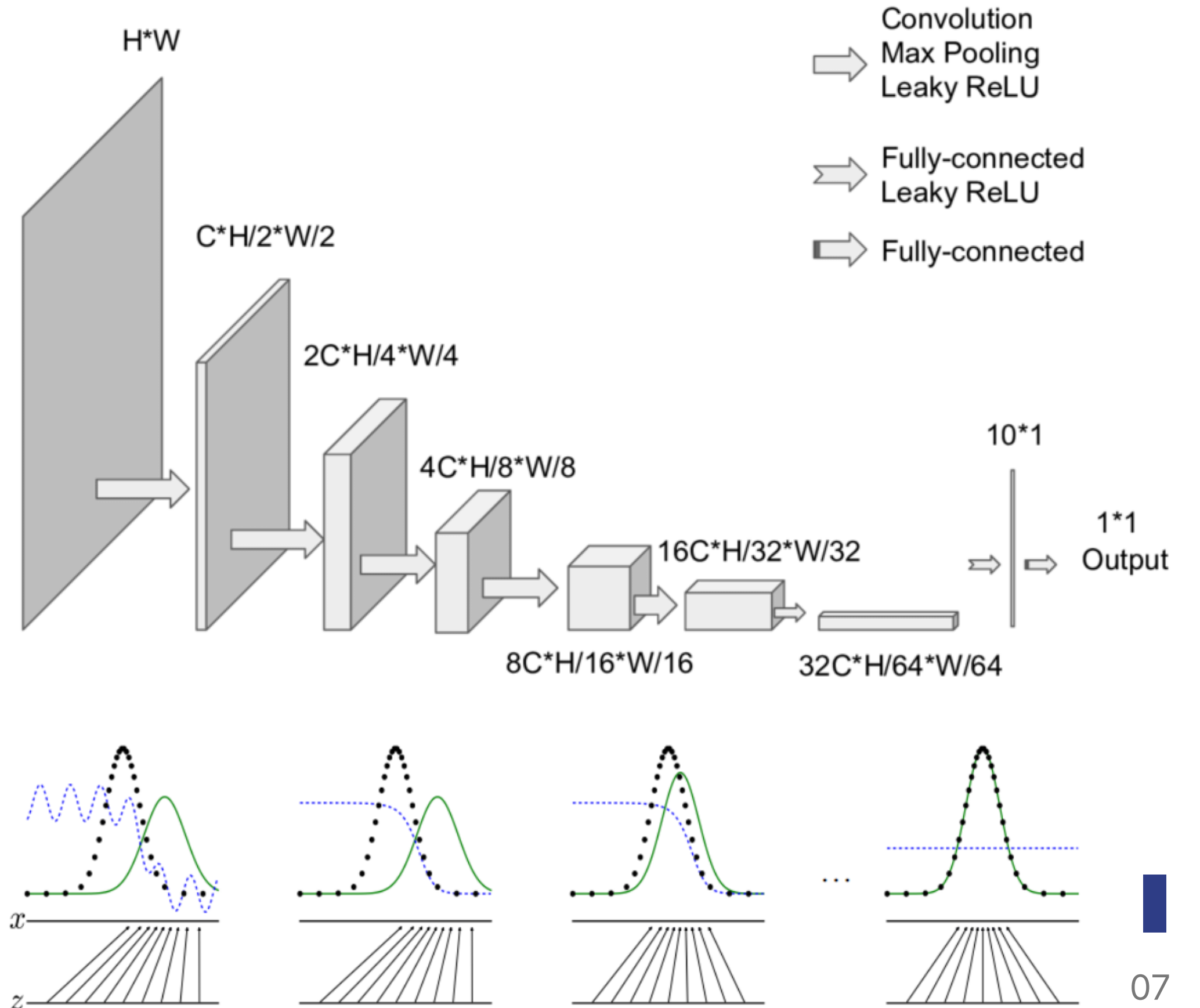
discriminator 내부의 신경망을 통과한 실제 데이터와 Generator가 생성한 데이터를 하나의 벡터 값으로 반환

2) Architecture(최근에 쓰이는 방향)

일반적인 신경망으로 convolution 레이어를 통과하고 Flatten 후 1*1 로 BCE

3) discriminator를 통과한 실제 데이터는 BCE를 통과하기 때문에 1에 가까운 어떤 수를 생성해야 함. 반면, discriminator를 통과한 Generator 데이터는 BCE를 통과하면 0에 가까워야 함

vector 값에 의해 계산된 loss를 보고 생성된 이미지가 어느 방향으로 학습되어야 하는지 알수 있음



GAN

(Generative Adversarial Network/생성형 대립 신경망)

Loss

1) discriminator loss
기본적인 Binary Cross-entropy

2) generator loss
BCE의 실제 데이터 부분

3) Adversarial
discrimination loss와 generator loss가 경쟁하듯 가중치를 갱신해가면서 훈련하는 방식을 '적대적' 혹은 '대립적'이라고 함
discriminator와 generator는 훈련과정에서 둘 다 강화되면서 점점 Real data와 Fake data가 구분 불가능해짐

참고: 경사하강법을 사용하기 위해서 loss는 0으로 감소하여야 하므로 앞에 -가 붙어야 함. 그러나 설명의 편의를 위해서 -를 생략함.

$$\mathbb{E}_{real} [\log D(x)] + \mathbb{E}_{fake} [\log(1 - D(x))]$$

$$BCE = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

Discriminator Loss $D(x)$
 $\log(D(x))$
Real data Loss

이진 분류로 생각할 때,
Real data의 $y = 1$
실제 데이터이기 때문에 + 이후 식은 0
1을 만들기 위해서
 $\log(D(x)) \rightarrow$ 최댓값

$\log(1 - D(G(z)))$
Fake data loss
이진 분류로 생각할 때,
Fake data의 $y = 0$
생성 데이터이기 때문에 + 이전 식은 0
1을 만들기 위해서
 $\log(1 - D(x)) \rightarrow$ 최댓값
 $D(x) \rightarrow$ 최솟값

Generator Loss $G(z)$

Fake data의 Discriminator Loss를 Real data 처럼 검증하는 훈련
즉, $\log(D(x)) = 1$ 로 보내는 방향 $\rightarrow \log(D(G(z))) = 1$
 $G(x) \rightarrow$ 최댓값 [$G(x) = \log(p(z))$]

GAN

(Generative Adversarial Network/생성형 대립 신경망)

평가 지표

1) IS (Inception Score)

생성 이미지를 Inception-v3 분류기에 넣어 한 장당 예측이 뚜렷(confident)하고, 전체적으로 클래스가 골고루 생성(diverse)되었는지 확인 (클수록 좋은 지표)

2) FID (Fréchet Inception Distance)

실제 이미지와 생성 이미지를 Inception의 특징공간 (pool3, 2048차원)에 임베딩하고, 두 분포를 가우시안으로 근사해 평균/공분산 차이를 측정

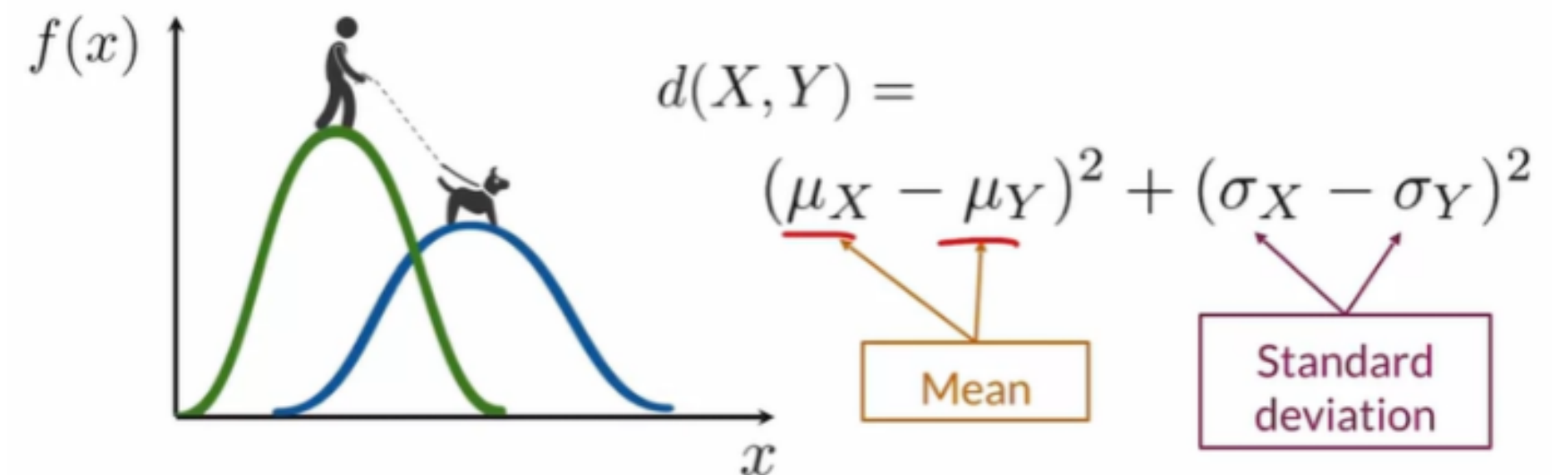
$$IS = \exp(\mathbb{E}_{x \sim p_\epsilon} D_{KL}(p(y | x) || p(y)))$$

KL Divergence

$p(y | x)$: 생성이미지 x 에 대한 Inception 분류 확률

$p(y) = \mathbb{E}_x[p(y | x)]$: 전체 생성샘플의 평균 클래스 분포

Fréchet Distance Between Normal Distributions



GAN

(Generative Adversarial Network/생성형 대립 신경망)

한계

1) 모드 콜랩스(mode collapse)

데이터 분포의 여러 모드를 덜 커버하고 몇 가지 패턴만 반복 생성 -> 다양성 저하

2) 평가와 재현성 문제

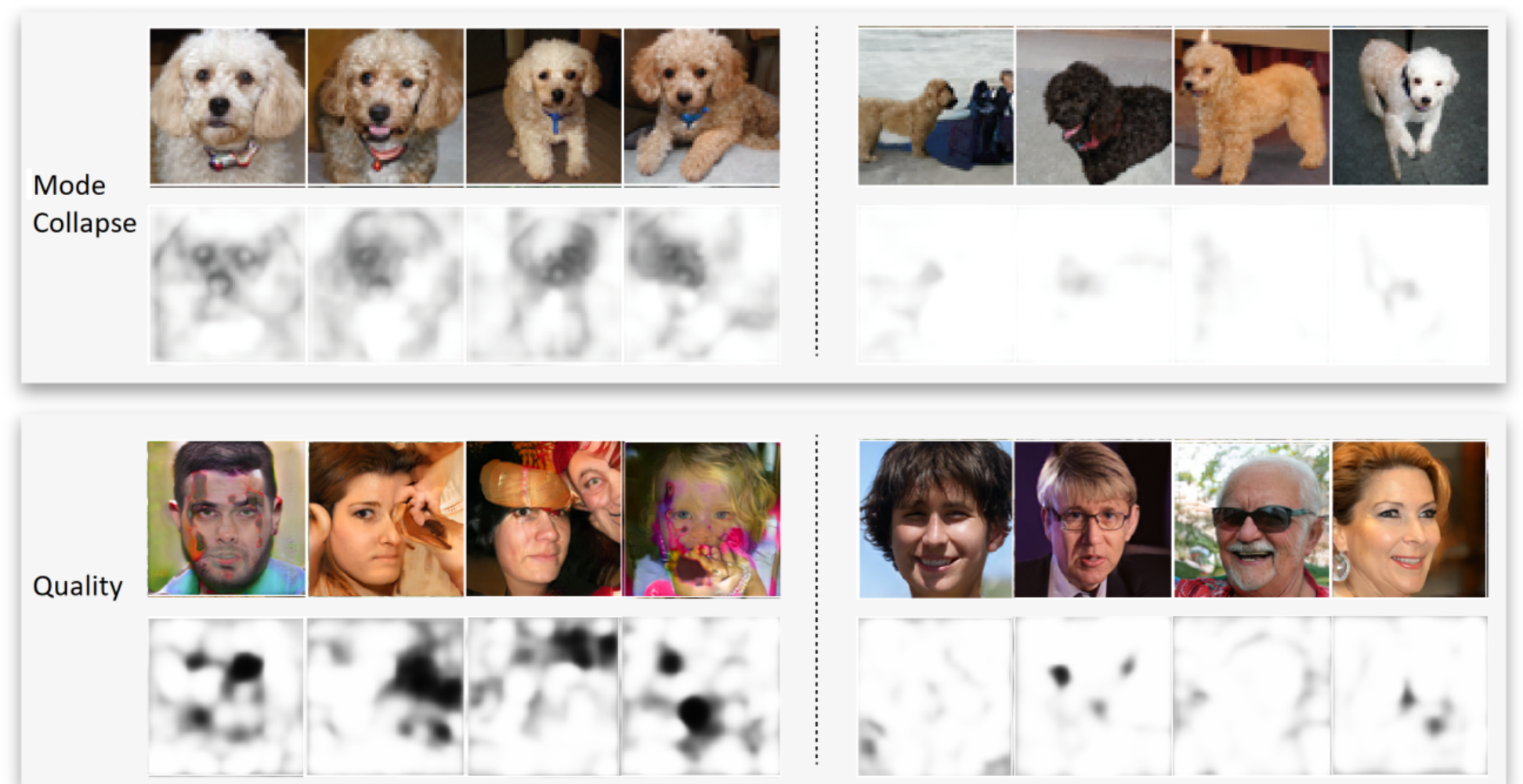
절대적인 척도가 없고 FID/IS가 구현·전처리·샘플수에 민감 -> 실험 재현이 어려움

3) discriminator 과강화/기울기 소실

optimal discriminator에 가까워지면 그라디언트가 0 근처로 소실 → 학습 정체.

4) 데이터 효율성과 편향

대형 고해상도 GAN은 많은 깨끗한 데이터가 필요



VAE

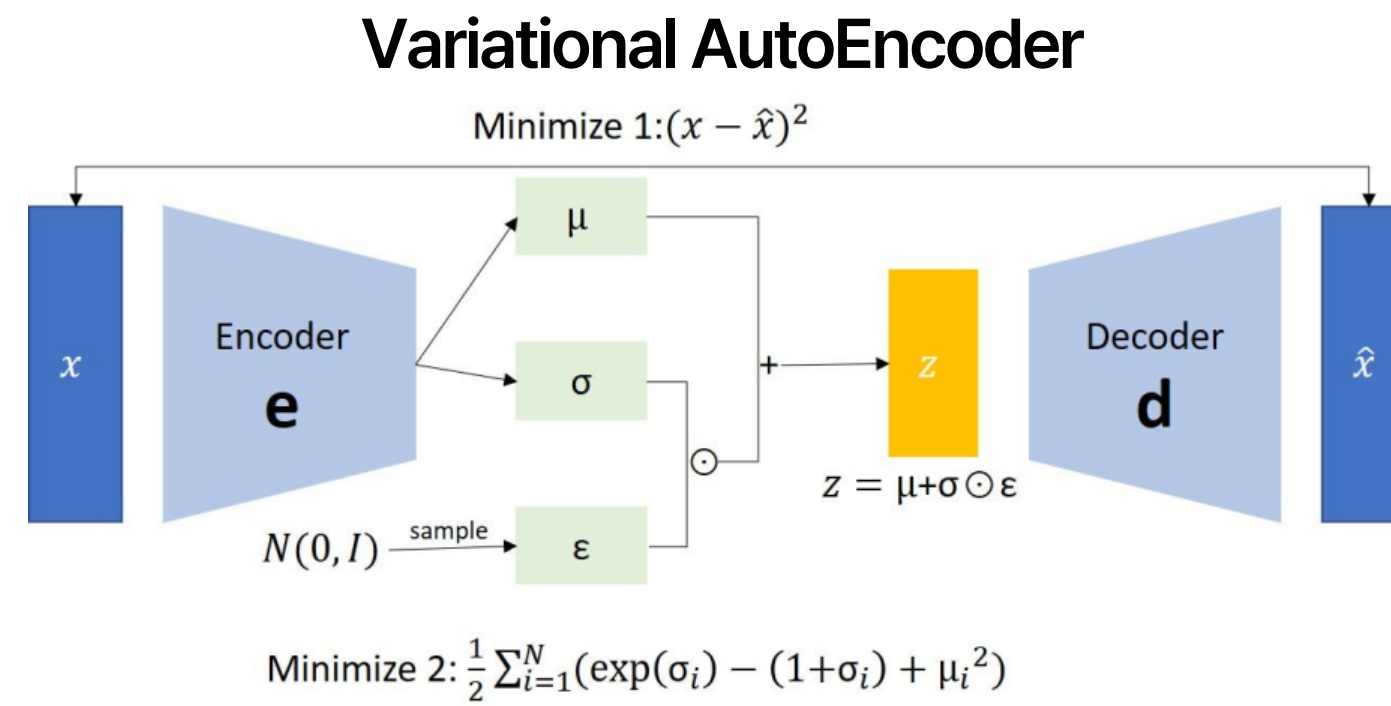
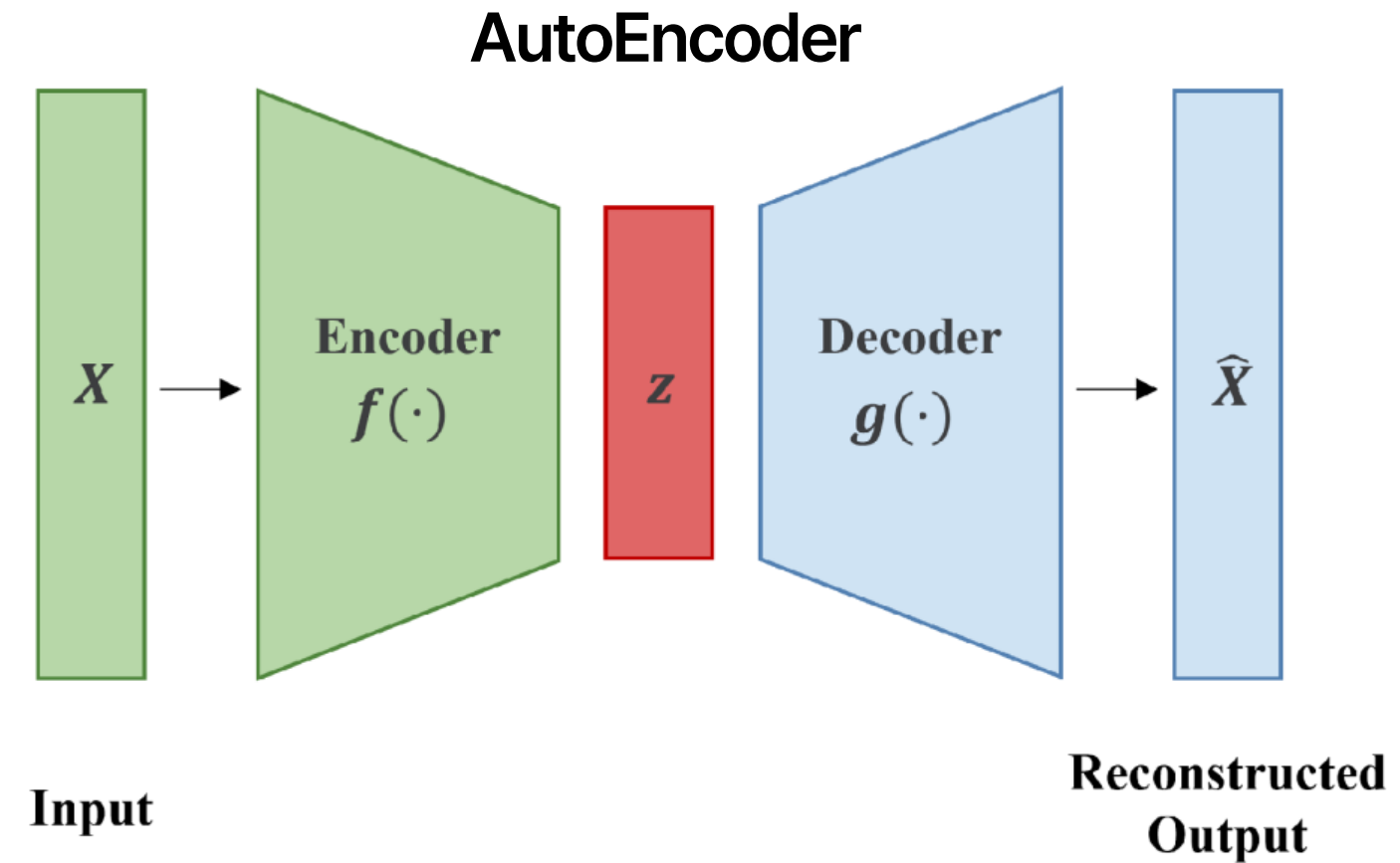
(Variational AutoEncoder / 확률적 오토 인코더)

개요

Encoder의 output으로 x 의 평균과 분산을 꺼내 사전 생성된 표준정규분포에서 샘플링한 값을 분산과 곱하고, 평균과 더해서 잠재 벡터(latent z)를 만들어냄
그 잠재 벡터를 decoder를 통과시켜 output을 꺼냄

간단 설명)

AutoEncoder에 표준정규분포 ϵ 를 샘플링하여 latent z 를 만드는 과정을 추가한 것



VAE

(Variational AutoEncoder / 확률적 오토 인코더)

Encoder

1) $q_{\phi}(x)$

어떤 x 를 넣어 잠재 벡터로 바꾸는 부분

2) Assumption(가정)

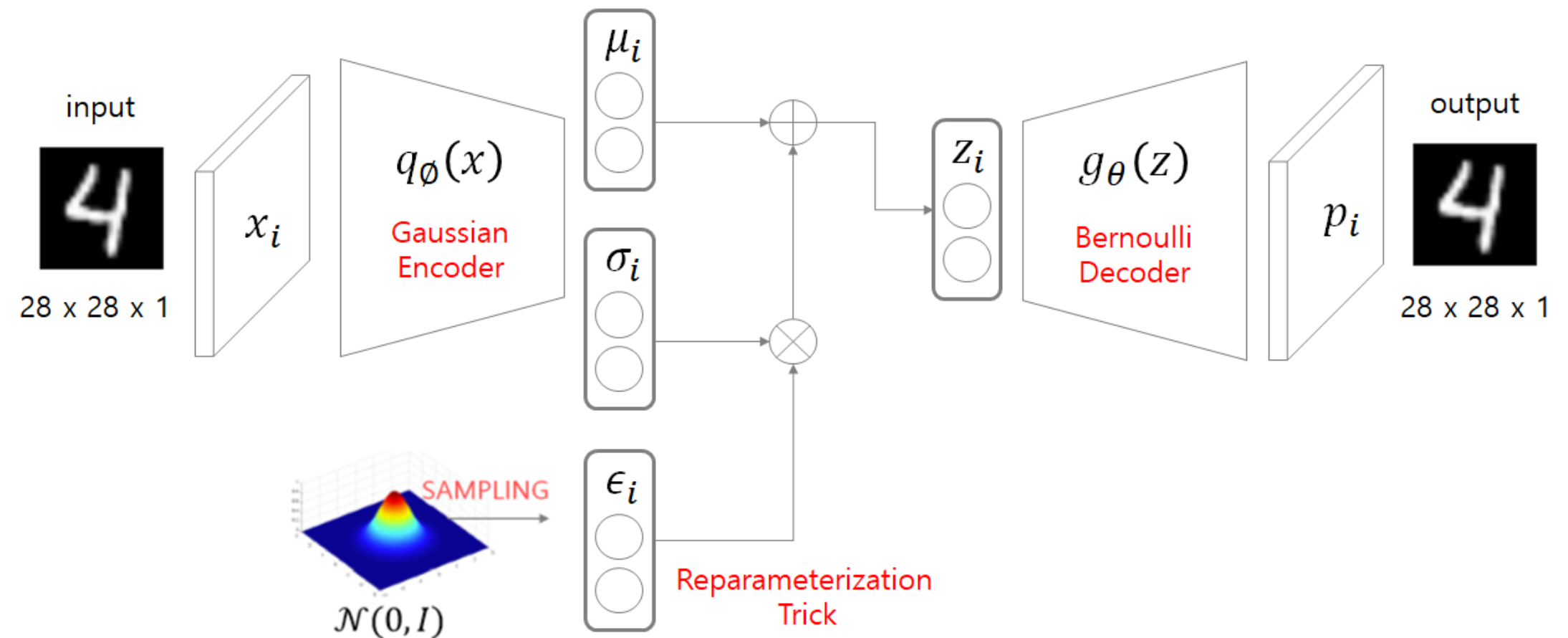
x 는 Gaussian Distribution(가우시안 정규분포)를 따름

3) μ_i / σ_i

평균과 분산을 output으로 각각 n 개 차원으로 산출

4) Reparameterization Trick(재파라미터화 트릭)

미분 불가능 지점 제거를 위해 적용



VAE

(Variational AutoEncoder / 확률적 오토 인코더)

Decoder

1) $p_\theta(x)$

잠재 벡터 z 가 들어가서 $\hat{x}$ (처음 input의 형태)를 산출하는 부분

2) Assumption

잠재 벡터 z 가 Bernoulli Distribution(베르누이 분포)을 따름 -> CE를 사용하기 위해서 가정

3) Loss

Reconstruction Error + Regularization Error

-> 수식에 대한 설명은 아래 링크를 참고

<https://www.youtube.com/watch?v=GbCAwVVKaHY>

Reconstruction Error의 앞부분을 ELBO(Evidence lower bound)라 함

Intractable to compute $p(x|z)$ for every z !

$$\text{Data likelihood: } p_\theta(x) = \int p_\theta(z)p_\theta(x|z)dz$$

Simple Gaussian prior

Decoder neural network

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z \sim q_\phi(z|x^{(i)})} [\log p_\theta(x^{(i)})] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z \left[\log \frac{p_\theta(x^{(i)}|z)p_\theta(z)}{p_\theta(z|x^{(i)})} \right]$$

Taking expectation wrt. z (using encoder network) will come in handy later

(Bayes' Rule)

$$p(z|x) = \frac{p(x|z)p(z)}{p(x)}$$

$$= \mathbf{E}_z \left[\log \frac{p_\theta(x^{(i)}|z)p_\theta(z)}{p_\theta(z|x^{(i)})} \frac{q_\phi(z|x^{(i)})}{q_\phi(z|x^{(i)})} \right] \quad (\text{Multiply by constant})$$

$$= \mathbf{E}_z [\log p_\theta(x^{(i)}|z)] - \mathbf{E}_z \left[\log \frac{q_\phi(z|x^{(i)})}{p_\theta(z)} \right] + \mathbf{E}_z \left[\log \frac{q_\phi(z|x^{(i)})}{p_\theta(z|x^{(i)})} \right] \quad (\text{Logarithms})$$

$$= \mathbf{E}_z [\log p_\theta(x^{(i)}|z)] - D_{KL}(q_\phi(z|x^{(i)}) || p_\theta(z)) + D_{KL}(q_\phi(z|x^{(i)}) || p_\theta(z|x^{(i)}))$$

참고

$$\mathbf{E}_{z \sim q_\phi(z|x^{(i)})} \left[\log \frac{q_\phi(z|x^{(i)})}{p_\theta(z)} \right] = \int \log \frac{q_\phi(z|x^{(i)})}{p_\theta(z)} q_\phi(z|x^{(i)}) dz$$

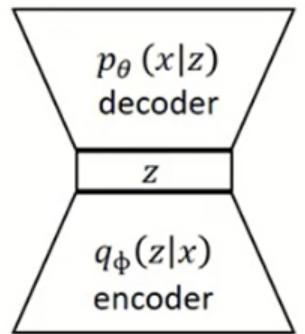
$$KL(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

The expectation wrt. z (using encoder network) let us write nice KL terms

$$\arg \min_{\theta, \phi} \sum_i -\mathbf{E}_{q_\phi(z|x_i)} [\log(p(x_i|g_\theta(z)))] + KL(q_\phi(z|x_i) || p(z))$$

Reconstruction Error

Regularization





02

Diffusion Model

Deep Unsupervised Learning Using Nonequilibrium Thermodynamics - Sohl-Dickstein et al.



Diffusion Model

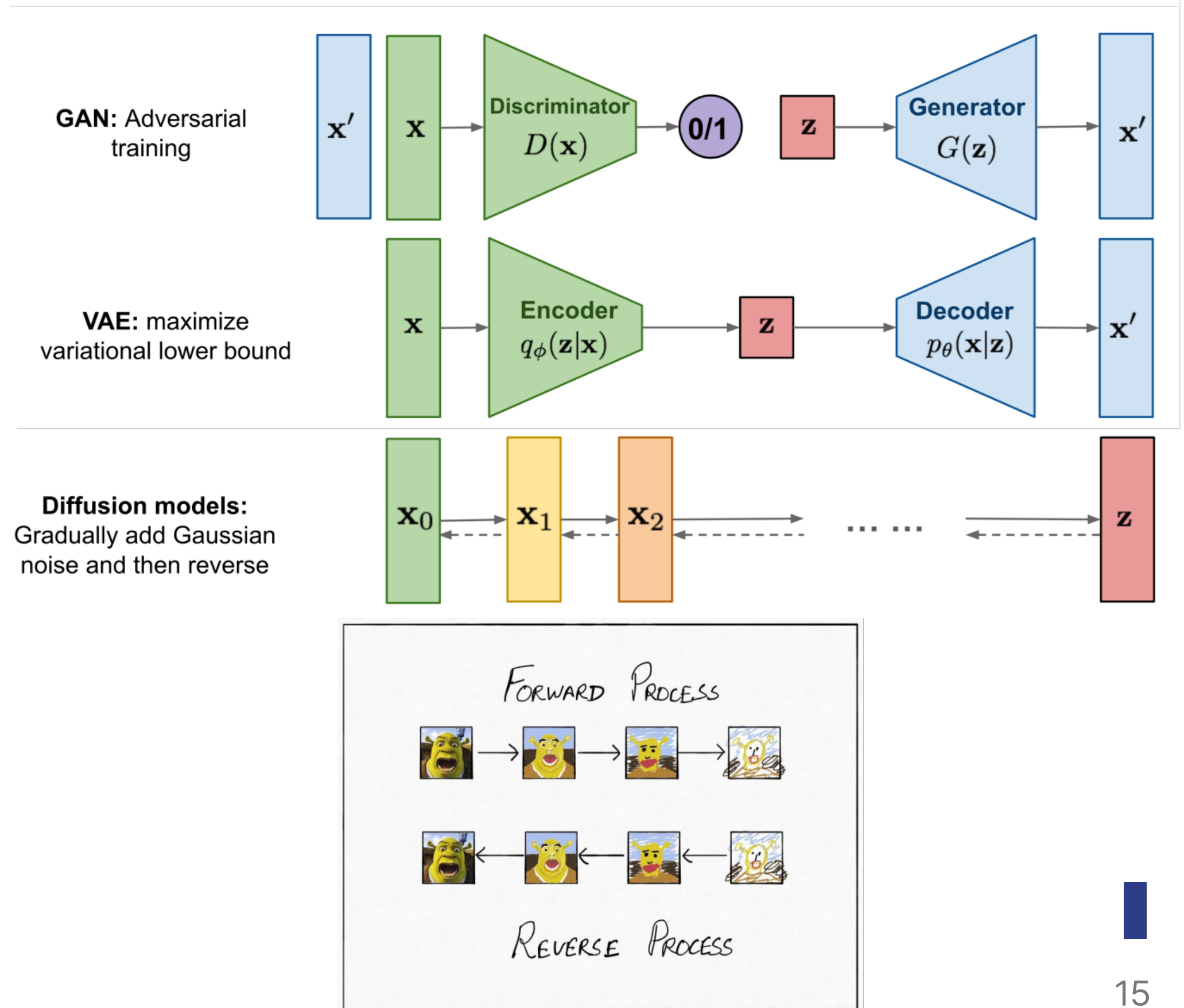
(확산 모델)

개요

어떤 이미지를 Gaussian Noise로 오염(Forward)시키고
그 과정을 반대로 학습해 오염을 제거하여(Reverse)원상
태로 복원하는 신경망
DDPM에서는 모든 Loss를 MSE로 만들어 단일 회귀
문제로 변환 -> 쉽고 정확한 학습

간단 설명)

input image에 일정한 간격으로 노이즈를 섞어서 완
전한 노이즈로 바꾸고, 다시 원상태로 노이즈를 걷어
내는 모델



Diffusion Model

(확산 모델)

Forward Process

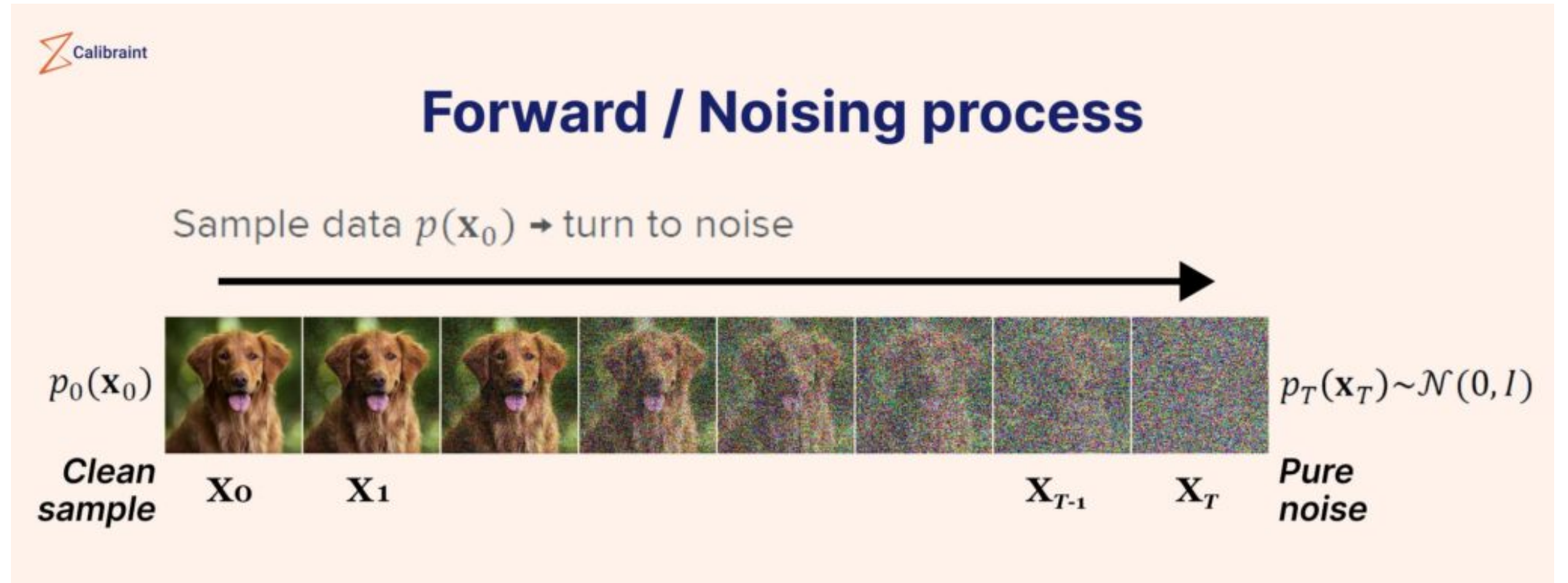
1) Gaussian Noise

원본 이미지에 step마다 noise를 추가

-> step t 에서 최종적으로 이미지 전부가 정규화된 noise로 변환

2) Markov Chain

바로 전 상태만 보면 다음 상태를 정할 수 있는 확률 과정
-> 어떤 step에서의 상태만 확인할 수 있으면 다음 상태를 예측할 수 있음. 따라서 한 상태의 noise의 양을 알 수 있음.(노이즈 스케줄(예: 선형, 코사인 등)은 훈련 내내 바뀌지 않고 고정)



Fixed Markov Chain

단일 과정 $q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad t = 1 \dots T$

누적 과정 $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s).$

샘플링 $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$

Diffusion Model

(확산 모델)

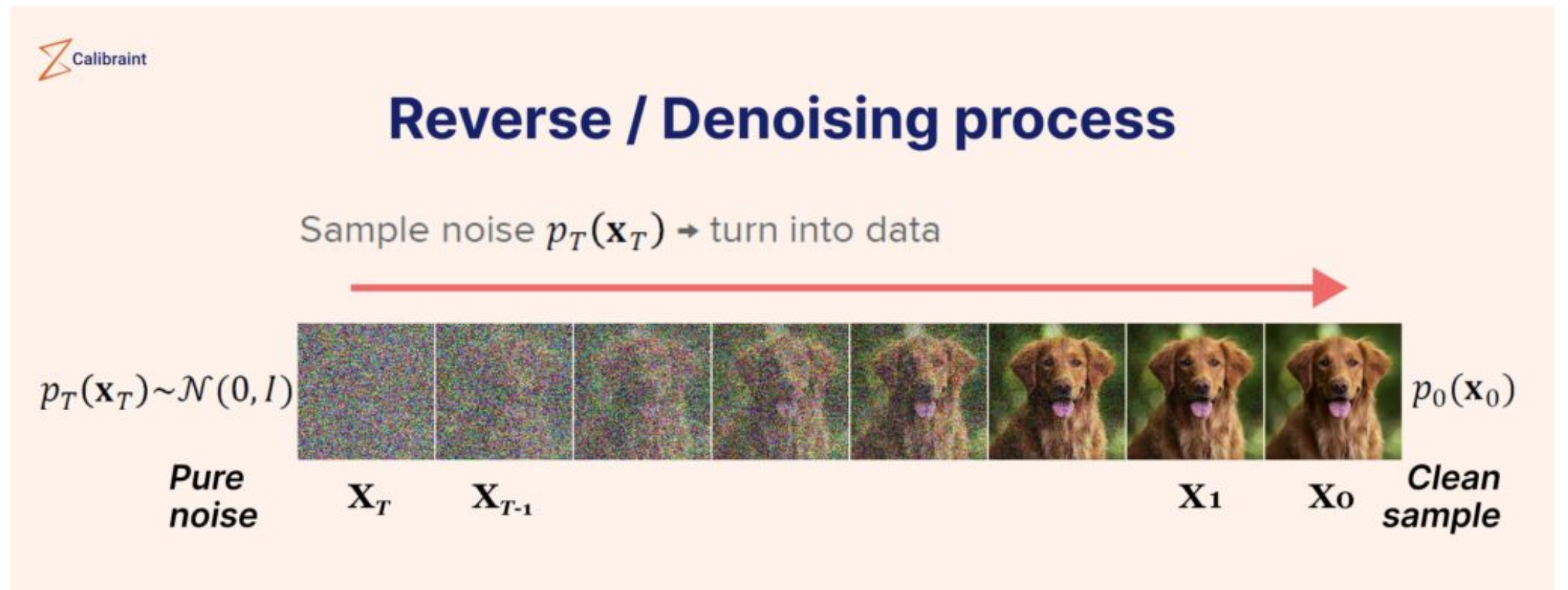
Reverse Process

1) 과정

노이즈가 끼있는 현재의 step t에서의 x를 학습하여 노이즈가 덜 낀 step t-1에서의 x의 평균을 예측
예측된 step t-1의 x의 평균에 약간의 가우시안 분포 값을 더함
step 0이 될 때까지 반복

2) 왜 가우시안 분포를 더할까?

평균만 맞추면 정답이 아님
역분포 또한 가우시안 분포를 따르고 있기 때문에 예측된 값도 가우시안 분포 형태로 수정해야 함



단일 과정 $p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(\mu_{\theta}(x_t, t), \sigma_t^2 I)$

누적 과정 $q(x_{t-1} | x_t, x_0) = \mathcal{N}(\underbrace{\tilde{\mu}(x_t, x_0)}_{\text{posterior mean}}, \underbrace{\tilde{\beta}_t I}_{\text{posterior var}})$

$$\tilde{\mu}(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{1 - \beta_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

$$\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$$



03

DDPM / DDIM

Denoising Diffusion Probabilistic Models - Jonathan Ho et al.

Denoising Diffusion Implicit Models - Jiaming Song et al.



DDPM

(Denoising Diffusion Probabilistic Model)

개요

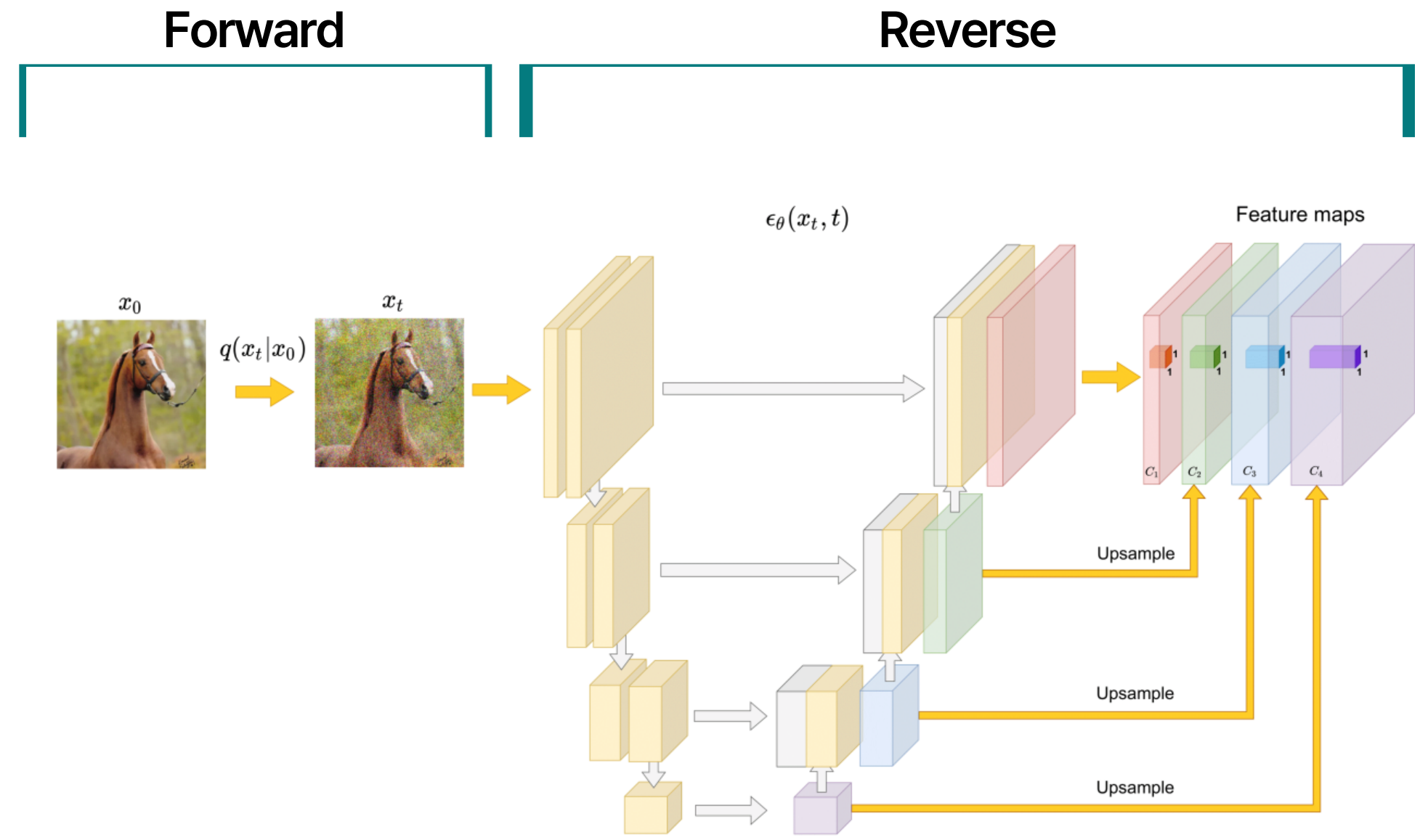
기본 diffusion model이 이전 step의 원형을 예측해서 찾는 방식으로 작동했다면, DDPM은 현재 step에서 노이즈의 양을 예측해서 제거했을 때, 수 차례 반복 후 step 0으로 찾아갈 수 있다는 아이디어

간단 설명)

Forward Process는 기본 diffusion model과 동일

Reverse Process는 아키텍처로 구성

-> 더 간단한 학습 · 더 좋은 샘플 · 큰 모델/데이터로의 확장 가능하게 만든 표준형



DDPM

(Denoising Diffusion Probabilistic Model)

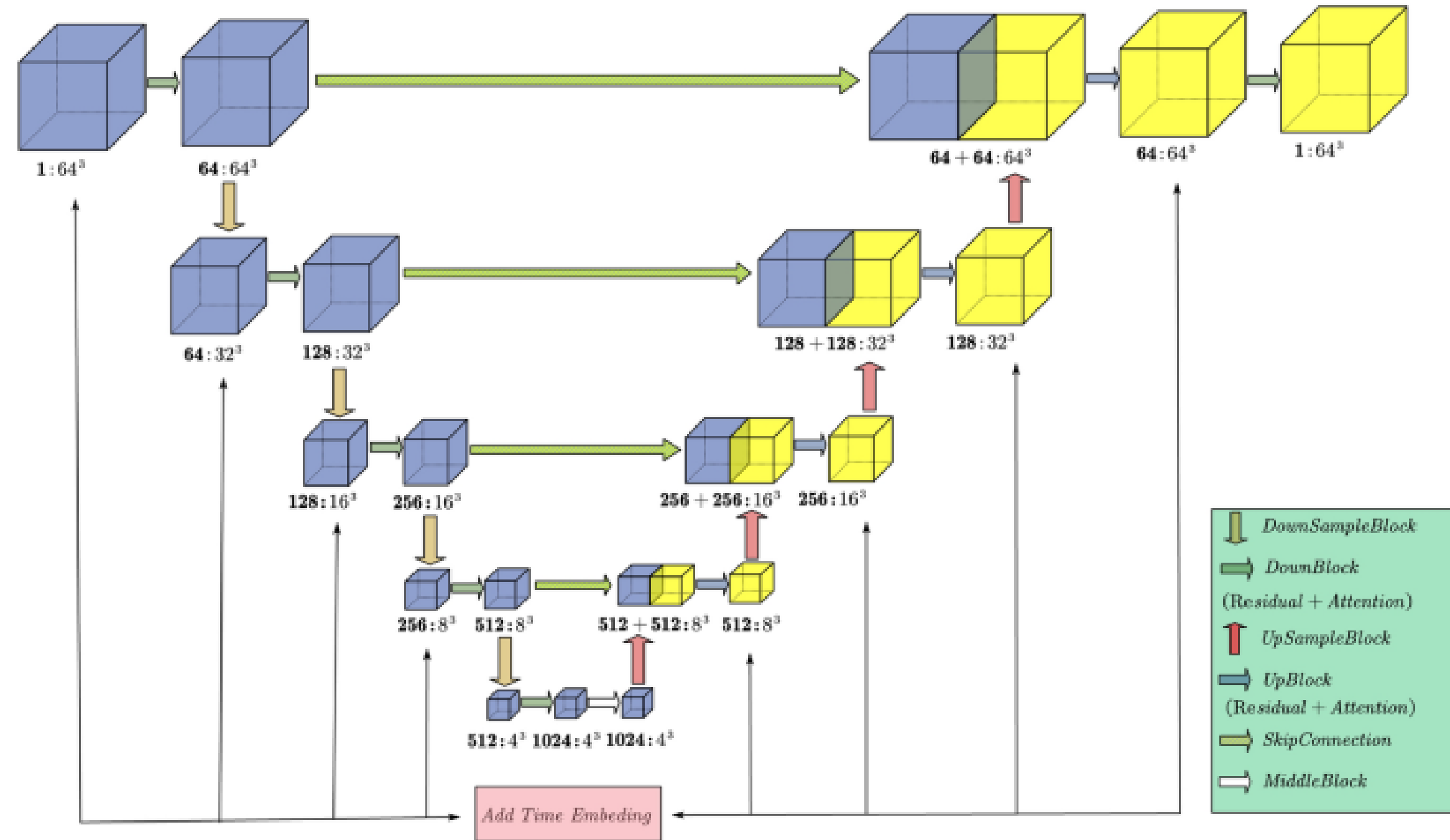
Reverse Process Architecture

1) U-net

- Contracting/Encoder: 다운샘플로 작은 해상도, 넓은 문맥 정보 획득
- Expanding/Decoder: 업샘플로 해상도 복구
- 가운데: bottleneck
- Skip Connection: 좌우 같은 단계끼리 Feature Maps 을 concat -> 위치 정보 보존
- > 구조가 U자처럼 보임

2) Residual Connection + Self-Attention

Residual Connection과 Self-Attention 이 매 Down Block, Up Block에서 발생
(원 논문은 Self-Attention / 이후 DDPM 변형 논문들은 Multi-head Self-Attention)



DDPM

(Denoising Diffusion Probabilistic Model)

Loss

1) 학습 할 때 t 는 1회

(t는 파라미터, 전통적으로 t=1000)

학습(train)의 경우 step마다 t를 1개 무작위 샘플링해서 forward-reverse 함 (Batch 당 t 1개를 샘플링한다는 뜻)

-> 모든 t를 학습하는 것이 아님

* MC(Monte-Carlo): 독립 난수를 무작위/대량으로 뽑아 평균을 내면 전체 평균에 근사할 수 있다는 이론

* MCMC(Markov Chain Monte-Carlo): 표본에서 무작위 난수로 분포를 뽑아 사용할 경우 평균과 분산에 근사됨 (추론 시에는 모든 t를 사용)

2) MSE 형태

MCMC에 의해 t-1 이 0으로 근사 되므로,
원본 noise - 예측 noise 의 mse형태로 근사

$$\begin{aligned}
 LOSS_{Diffusion} &= D_{KL}(q(z | x_0) || P_{\theta}(x_0 | z)) - E_{z \sim q(z|x)}[\log P_{\theta}(z)] \\
 &= D_{KL}(q(z | x_0) || P_{\theta}(z)) + \sum_{t=2} D_{KL}(q(x_{t-1} | x_t, x_0) || P_{\theta}(x_{t-1} | x_t)) - E_q[\log P_{\theta}(x_0 | x_1)]
 \end{aligned}$$

Regularizer on Encoder
Denoising Process
Reconstruction on Decoder

DDPM에서는 Loss가 굉장히 간단한 식으로 정의됨

$$LOSS_{DDPM} = \mathbb{E}_{x_0, \epsilon} \left[\left| \epsilon - \epsilon_{\theta} \left(\sqrt{\tilde{\alpha}_t} + \sqrt{1 - \tilde{\alpha}_t} \epsilon, t \right) \right|^2 \right]$$

MCMC에 의해 t를 한 개 샘플링하여 사용하기 때문 ←

DDIM

(Denoising Diffusion Implicit Models)

개요

DDPM과 train 과정에서의 Forward, Reverse는 동일
단, inference 과정에서의 Reverse과정에서 t 를 $\{t_s\}_{s=1}^S$
로 사용하여 추론 시간을 획기적으로 줄임
step의 t 마다 노이즈를 추가하는 DDPM과 반대로 노이즈 η 를 추가하지 않고 추론하는 모델

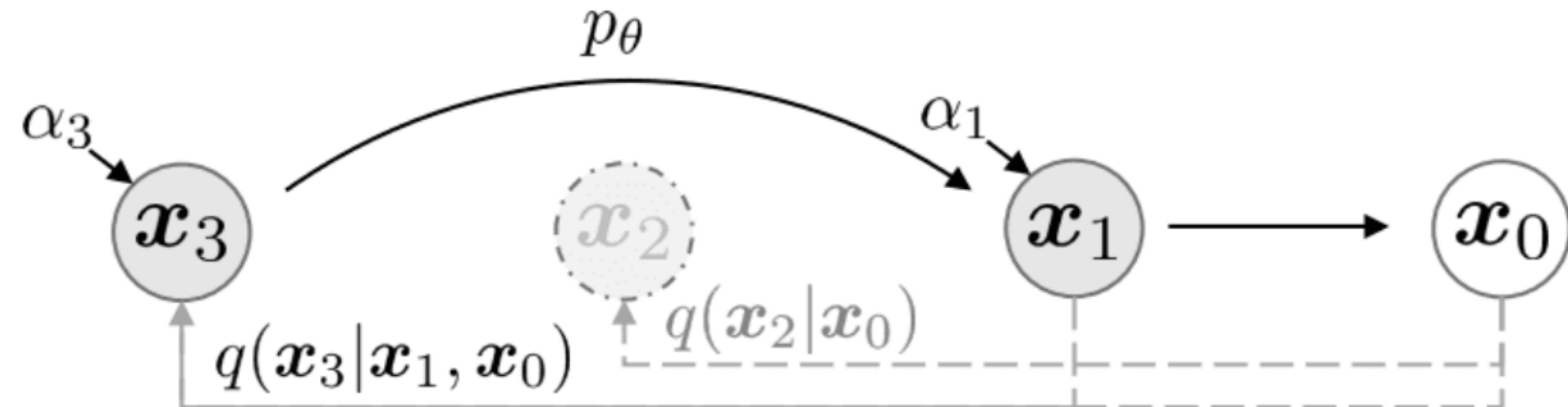
간단 설명)

test 파일을 추론할 때 t 를 샘플링하여 사용함

ex.

DDPM) $t = 1 \sim 1000 \rightarrow$ 추론시 $t = 1000$ 개

DDIM) $t = 1 \sim 1000 \rightarrow$ 추론시 t 를 50개 샘플링



DDIM

(Denoising Diffusion Implicit Models)

설명

1) 비(非) Markov Chain 샘플링

$\sigma_t z$, 의 잡음을 추가하지 않음

-> 잡음을 그냥 추가하지 않으면 분산이 0에 수렴하게 되어 성능이 망가짐

-> 대신 평균을 비 Markov Chain 으로 변경함

잡음항을 0으로 하되, 평균을 현재 t에서 예측된 x를 기반으로 최종 x를 계산하여 이를 기반으로 재구성

2) ϵ -pred

원본을 예측한 최종 x의 평균을 사용하더라도 DDIM의 목적은 t 마다 추가된 노이즈의 양을 예측하는 것이기 때문에 과적합 되지 않음

DDPM reverse

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t z, \quad z \sim \mathcal{N}(0, I), \quad \sigma_t^2 \approx \tilde{\beta}_t > 0$$

DDIM reverse

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \hat{\epsilon}_{\theta} \quad \hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_{\theta}}{\sqrt{\bar{\alpha}_t}},$$



04

Stable Diffusion

Prompt-to-Prompt Image Editing with Cross Attention Contro - Amir Hertz et al.

SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis - Dustin Podell et al.



Stable Diffusion

개요

Forward Process:

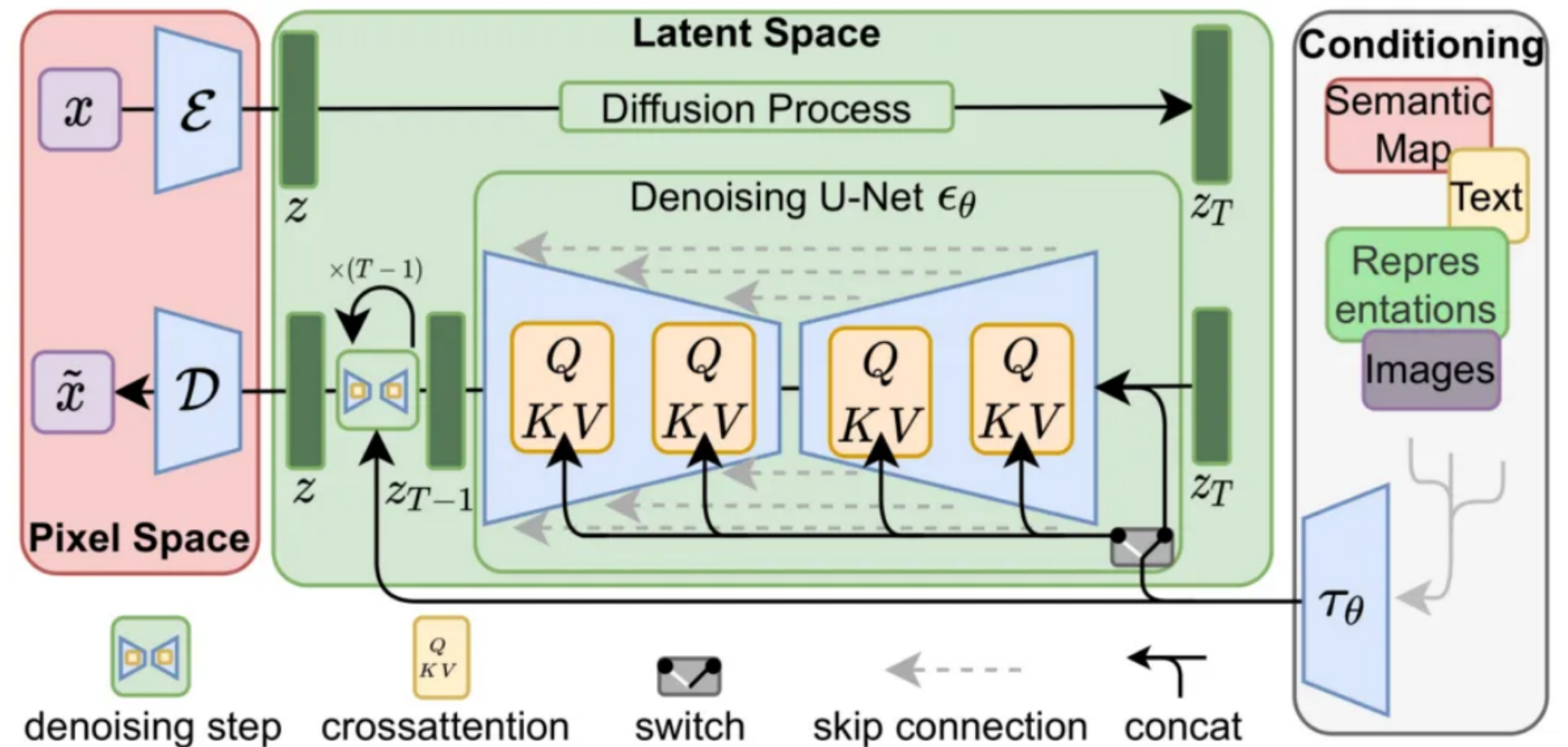
- 1) training set으로 x 를 넣고 VAE encoder를 통과 시켜 clean latent z 를 만듦
- 2) clean latent z 를 Forward process를 통과시켜 pure noise latent z 를 만듦

Reverse Process:

- 1) pure noise latent z 를 U-net을 통과 시킴
- 2) t 마다 U-net은 Cross-Attention으로 이루어져 있음
- 3) clean latent z 로 만드는 과정을 학습
- 4) VAE decoder 통과시켜서 최종 그림을 예측

Loss:

Latent: DDPM Loss / Codec: VAE Loss



Stable Diffusion

Cross-Attention

요약)

타깃(Q)이 소스(K/V)를 참조해서 정보를 끌어오는 연산

Self-Attention 과의 차이)

Self-Attention은 입력된 한 개의 행렬을 Q/K/V로 변환하여 연산

Cross-Attention은 입력된 두 개의 행렬을 목적 타깃 행렬(Q)과 참조하는 소스 행렬(K/V)로 변환하여 연산

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

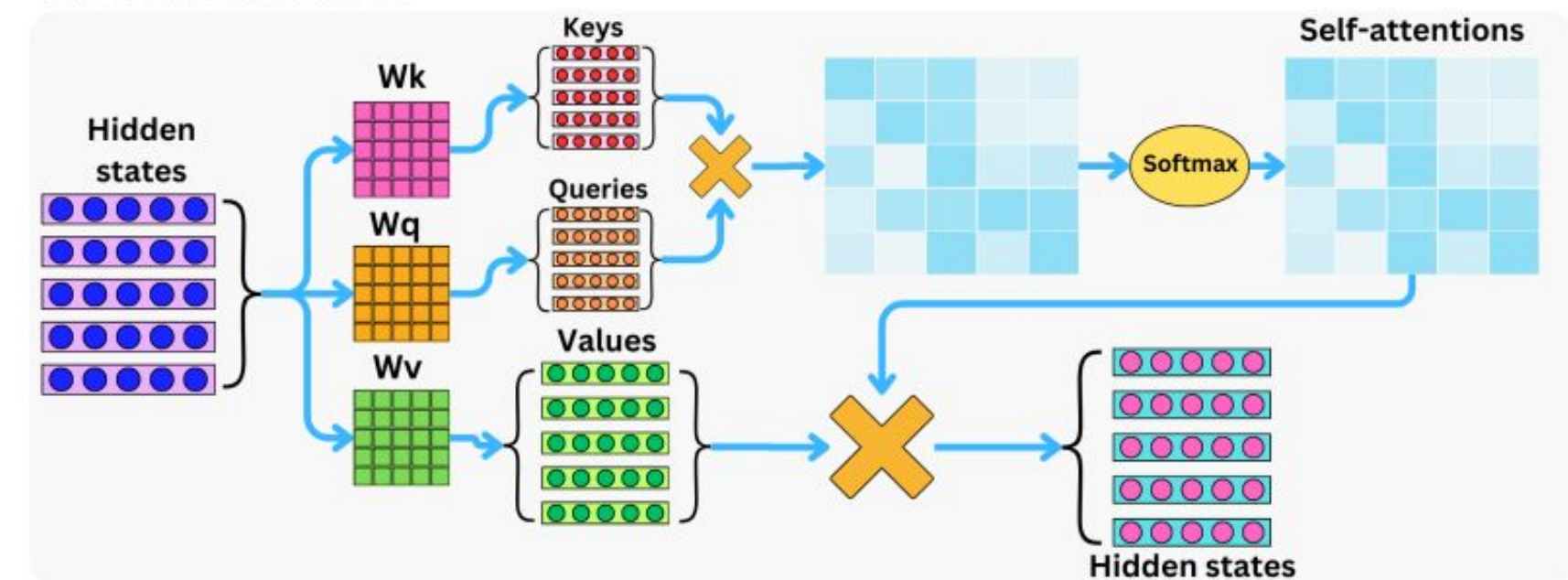
SA와 CA는 input의 개수가 다름

Stable Diffusion의 경우)

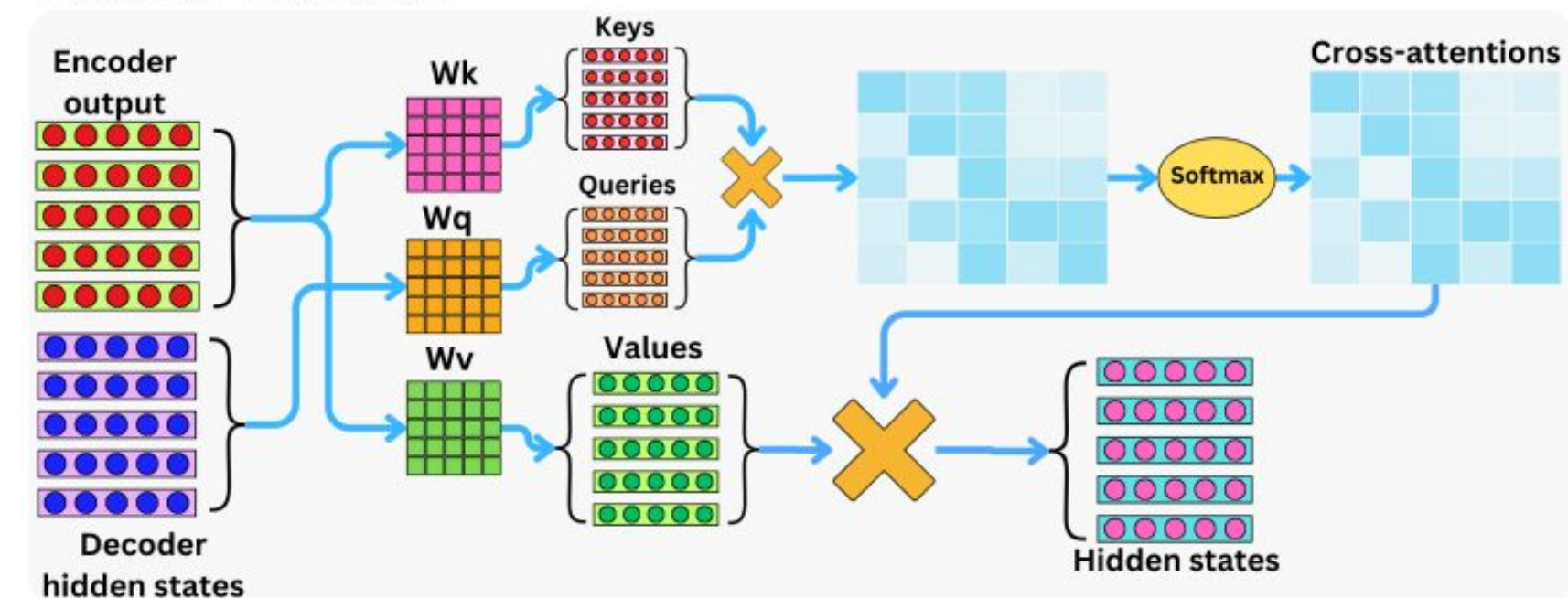
Q: pure noise latent z

K/V: Conditioning 된 벡터 (τ_θ)

The Self-Attentions



The Cross-Attentions



Stable Diffusion

Conditioning

1) 이미지

SD 1.x : CLIP ViT-L/14

SD 2.x : OpenCLIP ViT-H/14

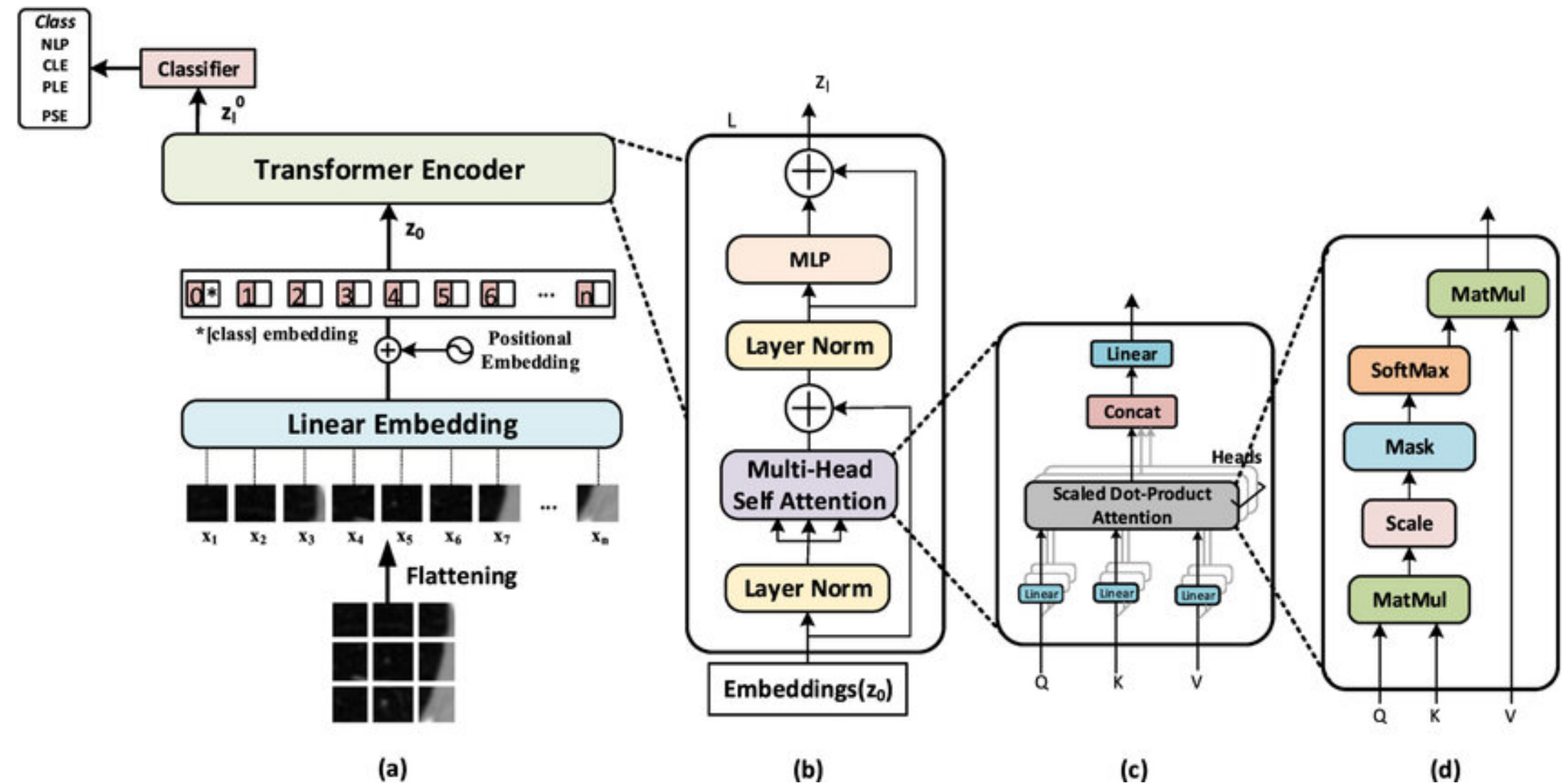
(Vision Transformer는 뒤에서 설명)

2) 텍스트

SD 1.x: CLIP 텍스트 인코더(ViT-L/14의 텍스트 타워)

SD 2.x: OpenCLIP 텍스트 인코더(ViT-H/14의 텍스트 타워)

흐름: BPE 토크나이저 → 임베딩 테이블 → 트랜스포머 → 텍스트 임베딩 → U-Net Cross-Attention의 K/V



Stable Diffusion

Vision Transformer

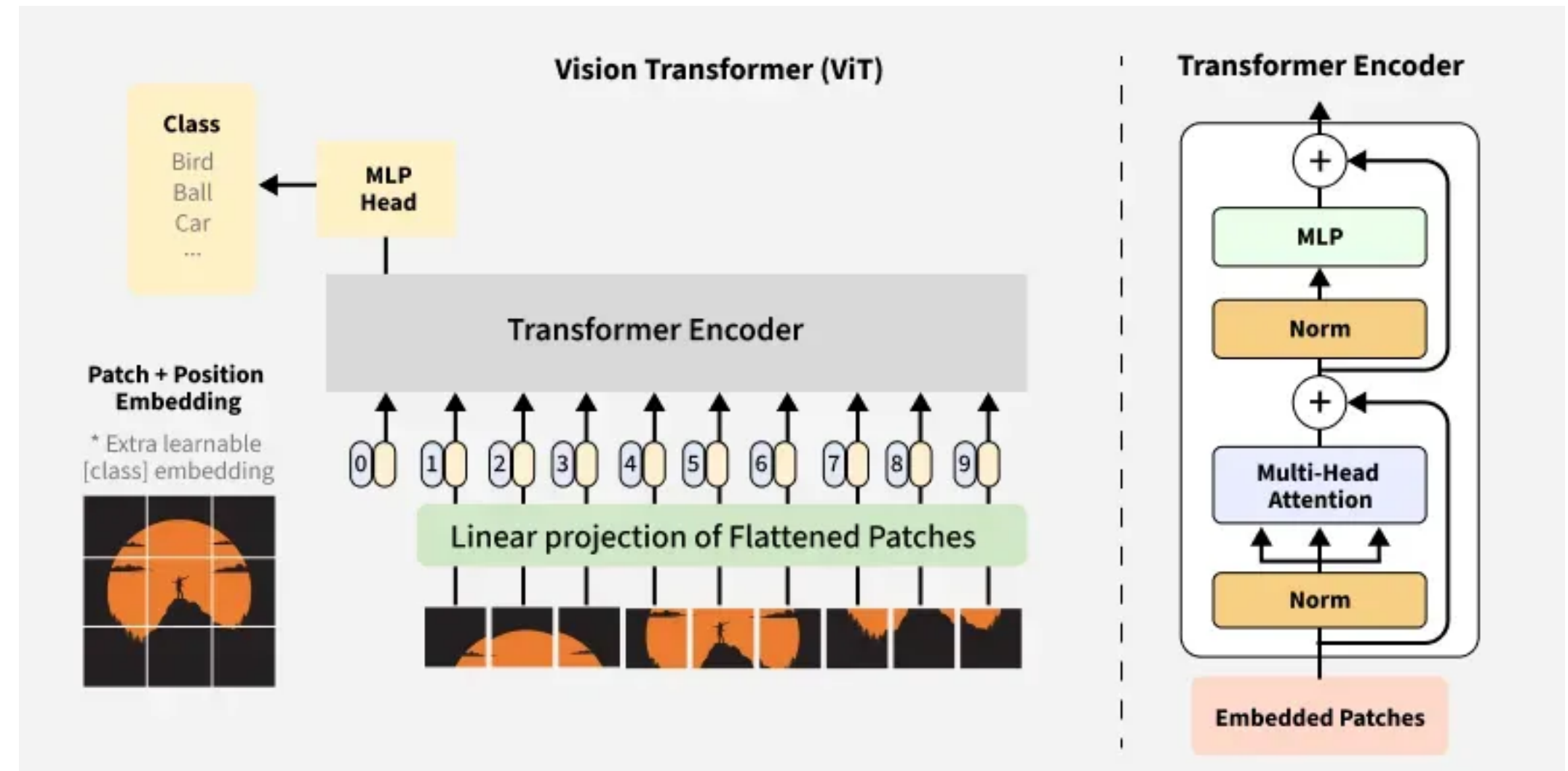
1) Patch 단위

픽셀 단위로 input 하지 않고 Patch라는 하이퍼 파라미터 단위로 나눠서 input(YOLOv1의 GridCell 처럼)
(보통 $\text{patches} = P * P$, $\text{stride} = P$)

2) Positional Embedding

Absolute PE: learned(각 위치마다 학습 가능한 벡터를 더함), Sinusoidal(고정 수식 기반 - $\sin$, $\cos$)
Relative PE: 토큰 간 거리/방향을 파라미터화해서 어텐션 점수에 직접 더함

3) Pre-Layer Normalization (뒤에서 따로 설명)



Stable Diffusion

Pre-LN / Post-LN

1) 구조:

Pre-LN: Norm $\rightarrow$ (process) $\rightarrow$ Add

Post-LN: (process) $\rightarrow$ Add $\rightarrow$ Norm

2) 사용

Pre-LN: 표준(ViT, 텍스트의 GPT류)

Post-LN: 최초 Transformer

3) 학습 안정성

Pre-LN: 잔차 경로가 순수한 원본 데이터에 가까워서 그 레디언트가 깊은 층까지 한번에 연결

- 깊은 네트워크/큰 LR에서도 안정적

Post-LN: 잔차 뒤에 LN이 있어서 순수 원본 경로가 끊김

- 깊어질수록 불안정/느린 수렴

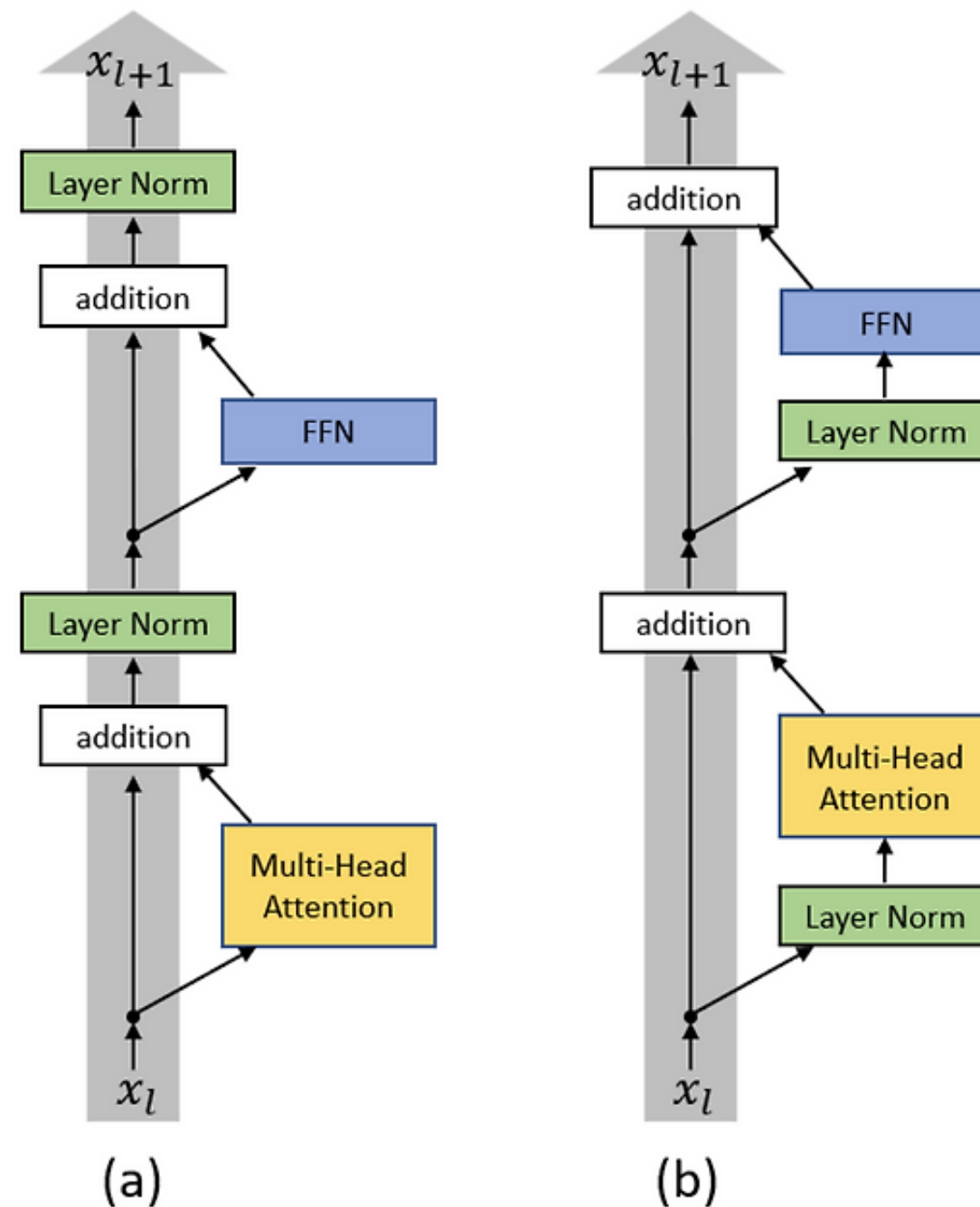
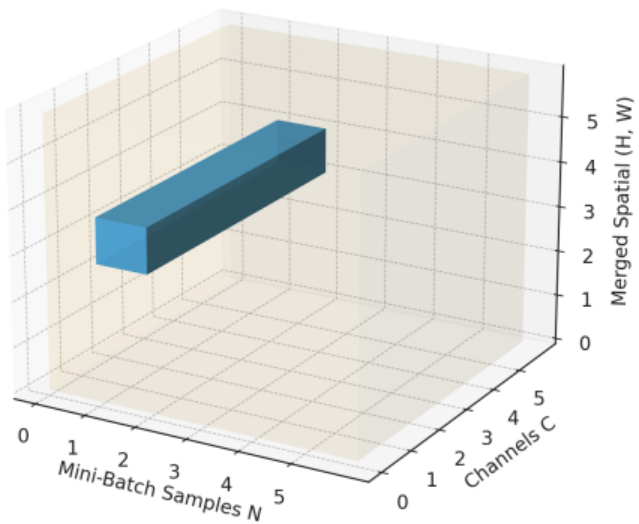
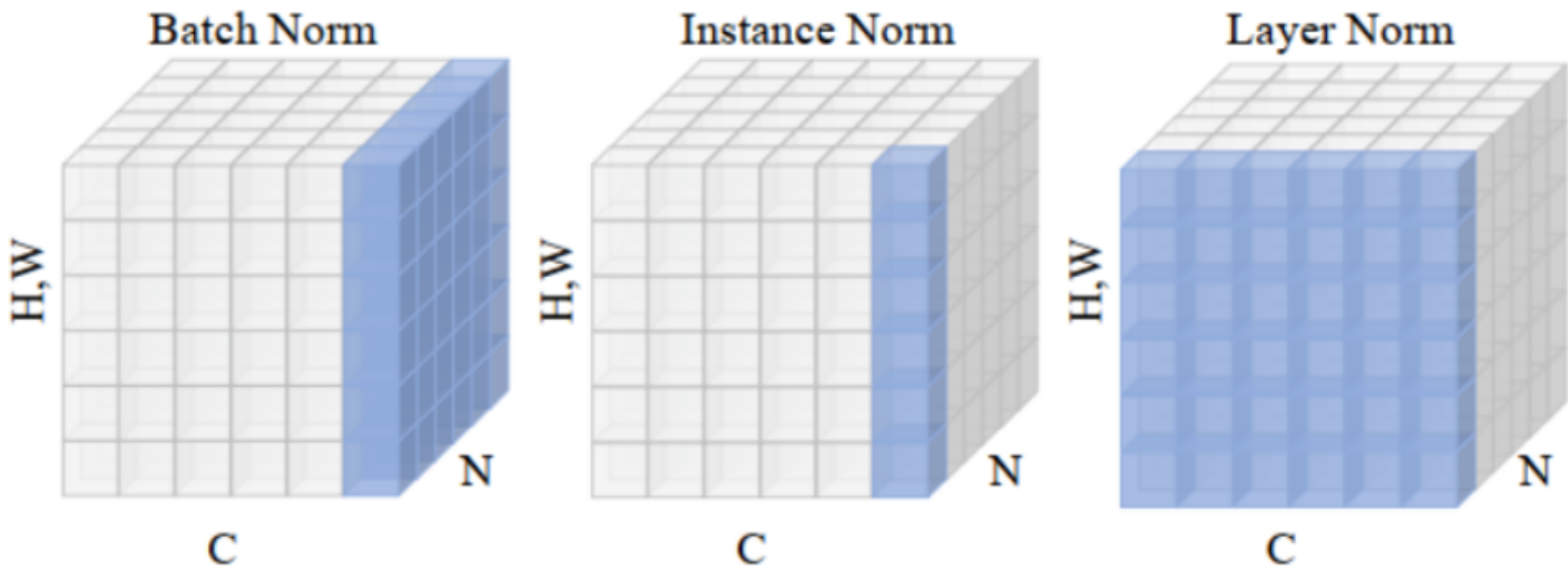


Figure 1. (a) Post-LN Transformer layer; (b) Pre-LN Transformer layer.

Stable Diffusion

Normalization



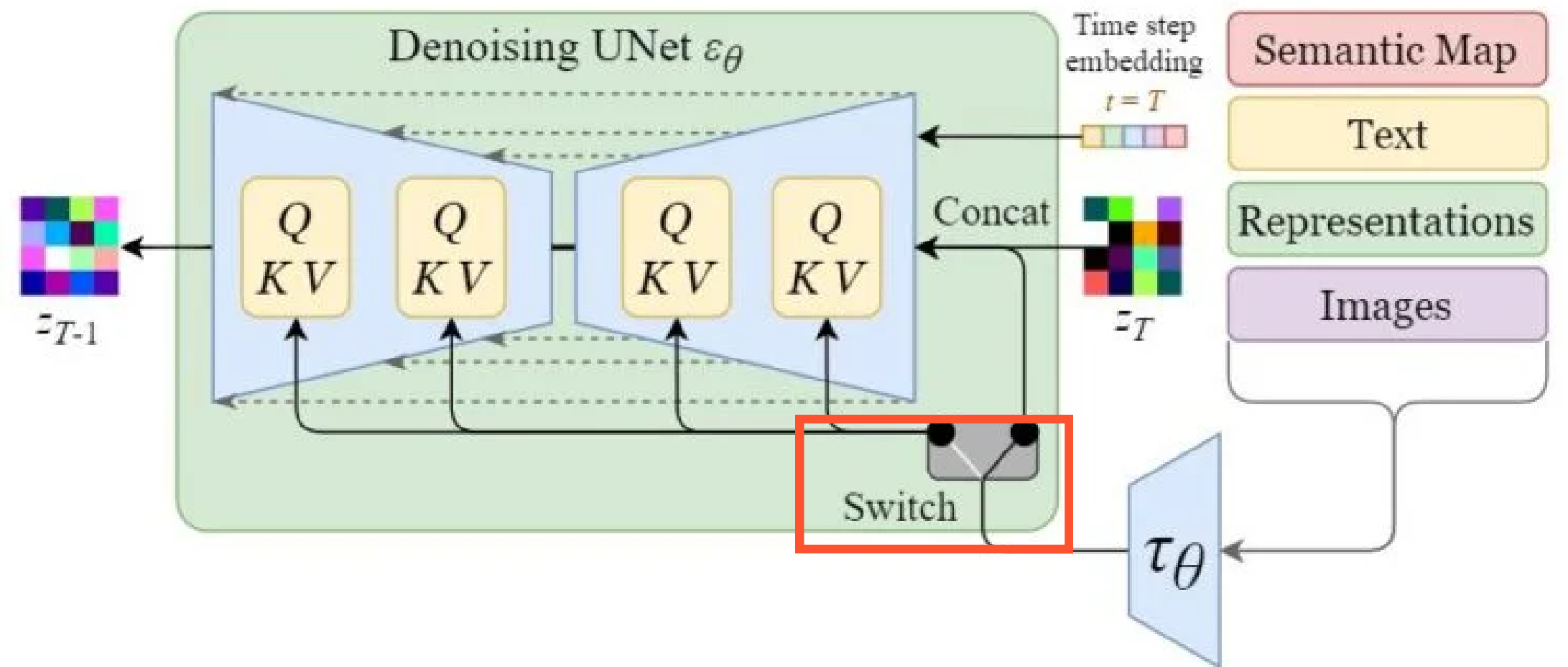
정규화 축 (Conv 기준)
기본(N,C,H,W)

Batch Normalization	각 채널별로 (N,H,W)	장점: 큰 배치에서 수렴 빠름, CNN 분류/검출에서 표준 단점: 작은/분산 배치에서 불안정, 학습/추론 불일치 이슈
Instance Normalization	각 샘플·채널별로 (H,W)	장점: 배치 무관 안정, 스타일/콘트라스트 제거에 특화(스타일 트랜스퍼, 생성 모델). 단점: 너무 많은 스타일 정보 제거 → 분류/검출에서 성능 하락 가능.
Layer Normalization	각 샘플의 (C,H,W)	장점: 배치 무관 안정, 트랜스포머/Pre-LN 구조에 주로 쓰임 단점: CNN 분류에선 BN보다 이득이 적을 수 있음
Pixel Normalization	각 샘플의 픽셀 위치 (H,W)에서 채널 C	장점: 간단, 배치 무관 안정, 스케일 불변. 초기 GAN에서 Generator 안정화에 사용. 단점: γ, β 가 없어 표현 유연성↓

Stable Diffusion

Switch Toggle

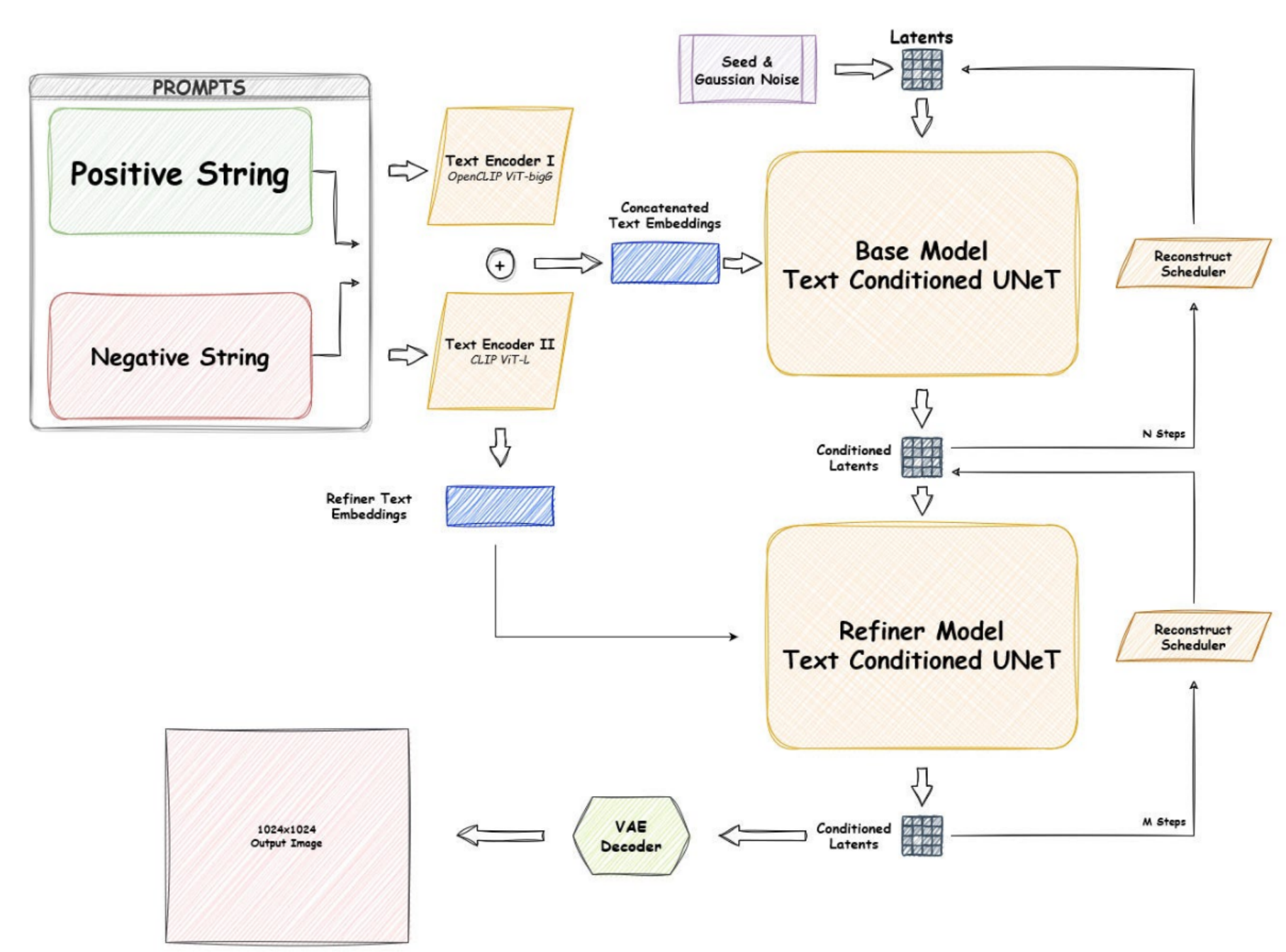
- 1) Conditioning 과정 이후 나온 값이 공간적 특성을 가진 벡터(Semantic Map, Image)면 입력되는 Q와 Concat
- 2) 텍스트 특성을 가진 벡터(Embedded Text, Embedded Category)면 K/V로 변환하여 Cross-Attention으로 입력



Stable Diffusion

Stable Diffusion XL

- 1) input으로 Conditioning 된 Prompt와 고수준 noise latent가 Base Model을 통과하여 저수준 noise latent를 산출
- 2) Reconstruct Scheduler에 따라 Base Model의 t를 역확산 루프로 학습
- 3) 저수준 noise latent와 Conditioning 된 Prompt를 Refiner Model을 통과하여 Construct latent를 산출
- 4) Reconstruct Scheduler에 따라 Refiner Model의 t를 역확산 루프로 학습
- 5) VAE decoder로 최종 이미지 산출





05

Code Implement

GAN/VAE/DDPM/Stable Diffusion XL



GAN 구현

(Generative Adversarial Network)

Data Set - Cifar10

Code

1) Generator Network

2) Discriminator Network

1) Generator Network

```
# Generator 네트워크
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.main = nn.Sequential(
            # 입력: latent_dim x 1 x 1
            nn.ConvTranspose2d(100, 512, 4, 1, 0, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(True),
            # 512 x 4 x 4
            nn.ConvTranspose2d(512, 256, 4, 2, 1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(True),
            # 256 x 8 x 8
            nn.ConvTranspose2d(256, 128, 4, 2, 1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            # 128 x 16 x 16
            nn.ConvTranspose2d(128, 3, 4, 2, 1, bias=False),
            nn.Tanh()
            # 3 x 32 x 32
        )

    def forward(self, x):
        return self.main(x)
```

2) Discriminator Network

```
# Discriminator 네트워크
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            # 입력: 3 x 32 x 32
            nn.Conv2d(3, 128, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # 128 x 16 x 16
            nn.Conv2d(128, 256, 4, 2, 1, bias=False),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2, inplace=True),
            # 256 x 8 x 8
            nn.Conv2d(256, 512, 4, 2, 1, bias=False),
            nn.BatchNorm2d(512),
            nn.LeakyReLU(0.2, inplace=True),
            # 512 x 4 x 4
            nn.Conv2d(512, 1, 4, 1, 0, bias=False),
            nn.Sigmoid()
        )

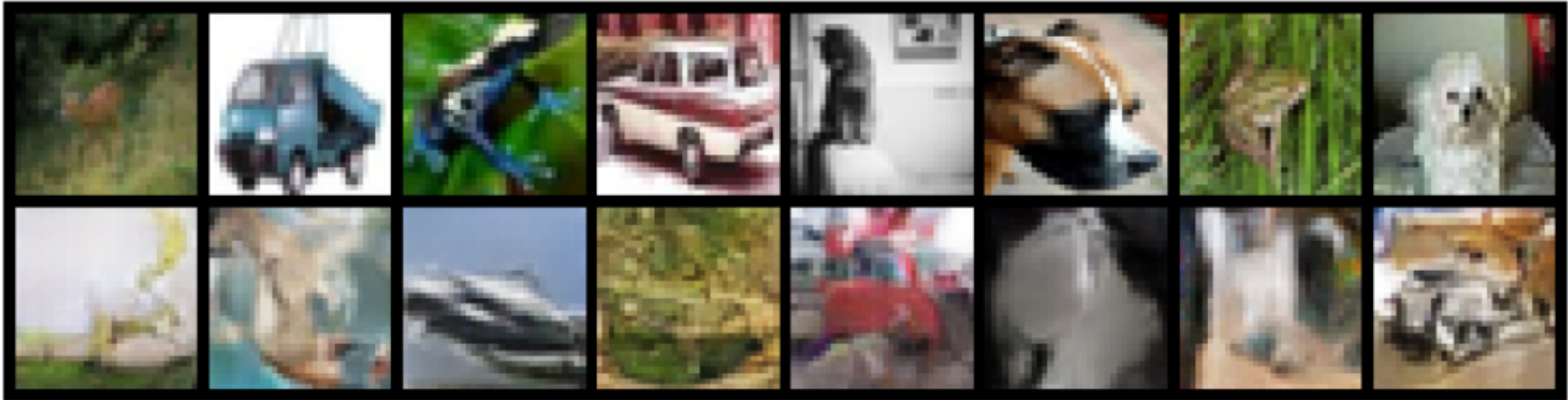
    def forward(self, x):
        return self.main(x).view(-1, 1).squeeze(1)
```


GAN 구현

(Generative Adversarial Network)
Data Set - Cifar10

- Code
- 1) Comparison
- 2) Loss

1) Comparison (Top: Real, Bottom: Generated)



2) Loss

```
49/50][0/391] Loss_D: 0.8768 Loss_G: 5.4501 D(x): 0.9434 D(G(z)): 0.4812/0.0089
49/50][100/391] Loss_D: 0.3946 Loss_G: 2.9304 D(x): 0.8993 D(G(z)): 0.2162/0.0773
49/50][200/391] Loss_D: 0.2604 Loss_G: 3.3614 D(x): 0.9430 D(G(z)): 0.1650/0.0481
49/50][300/391] Loss_D: 0.4752 Loss_G: 1.9369 D(x): 0.7159 D(G(z)): 0.0702/0.2090
```



VAE 구현

(Variational Auto-Encoder)
Data Set - Cifar10

Code

- 1) Encoder
- 2) Decoder
- 3) reparameterize trick

1) Encoder

```
class VAE(nn.Module):
    def __init__(self, latent_dim=128):
        super(VAE, self).__init__()

        # Encoder
        self.encoder = nn.Sequential(
            # 3 x 32 x 32
            nn.Conv2d(3, 32, 4, 2, 1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            # 32 x 16 x 16
            nn.Conv2d(32, 64, 4, 2, 1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            # 64 x 8 x 8
            nn.Conv2d(64, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            # 128 x 4 x 4
            nn.Conv2d(128, 256, 4, 2, 1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            # 256 x 2 x 2
        )

        self.fc_mu = nn.Linear(256 * 2 * 2, latent_dim)
        self.fc_logvar = nn.Linear(256 * 2 * 2, latent_dim)
```

2) Decoder

```
self.fc_decode = nn.Linear(latent_dim, 256 * 2 * 2)

self.decoder = nn.Sequential(
    # 256 x 2 x 2
    nn.ConvTranspose2d(256, 128, 4, 2, 1),
    nn.BatchNorm2d(128),
    nn.ReLU(),
    # 128 x 4 x 4
    nn.ConvTranspose2d(128, 64, 4, 2, 1),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    # 64 x 8 x 8
    nn.ConvTranspose2d(64, 32, 4, 2, 1),
    nn.BatchNorm2d(32),
    nn.ReLU(),
    # 32 x 16 x 16
    nn.ConvTranspose2d(32, 3, 4, 2, 1),
    nn.Tanh()
    # 3 x 32 x 32
)
```

3) reparameterize trick

```
def reparameterize(self, mu, logvar):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mu + eps * std
```


VAE 구현

(Variational Auto-Encoder)
Data Set - Cifar10

- Code
- 1) Comparison

2) Loss

1) Comparison



2) Loss

```
[49/50][0/391] Loss: 179.0816 Recon: 118.2935 KLD: 60.7881
[49/50][100/391] Loss: 183.5850 Recon: 123.6233 KLD: 59.9617
[49/50][200/391] Loss: 180.0448 Recon: 119.4071 KLD: 60.6376
[49/50][300/391] Loss: 184.6513 Recon: 124.7511 KLD: 59.9002
Epoch [49/50] Average Loss: 186.1374
```

DDPM

(Denoising Diffusion Probabilistic Model)

Data Set : UTKFace

Code

```
class UNet(nn.Module):
    def __init__(self, img_channels=3, time_dim=256):
        super().__init__()
        self.time_embed = TimeEmbedding(time_dim)

        # Encoder
        self.enc1 = nn.Sequential(
            nn.Conv2d(img_channels, 64, 3, padding=1),
            ResBlock(64, 64, time_dim * 4),
            ResBlock(64, 64, time_dim * 4)
        )
        self.down1 = nn.Conv2d(64, 64, 4, 2, 1)

        self.enc2 = nn.Sequential(
            ResBlock(64, 128, time_dim * 4),
            ResBlock(128, 128, time_dim * 4)
        )
        self.down2 = nn.Conv2d(128, 128, 4, 2, 1)

        self.enc3 = nn.Sequential(
            ResBlock(128, 256, time_dim * 4),
            ResBlock(256, 256, time_dim * 4)
        )
        self.down3 = nn.Conv2d(256, 256, 4, 2, 1)

        # Bottleneck
        self.bottleneck = nn.Sequential(
            ResBlock(256, 512, time_dim * 4),
            ResBlock(512, 512, time_dim * 4)
        )
```

```
# Decoder
self.up3 = nn.ConvTranspose2d(512, 256, 4, 2, 1)
self.dec3 = nn.Sequential(
    ResBlock(512, 256, time_dim * 4),
    ResBlock(256, 256, time_dim * 4)
)

self.up2 = nn.ConvTranspose2d(256, 128, 4, 2, 1)
self.dec2 = nn.Sequential(
    ResBlock(256, 128, time_dim * 4),
    ResBlock(128, 128, time_dim * 4)
)

self.up1 = nn.ConvTranspose2d(128, 64, 4, 2, 1)
self.dec1 = nn.Sequential(
    ResBlock(128, 64, time_dim * 4),
    ResBlock(64, 64, time_dim * 4)
)

self.final = nn.Conv2d(64, img_channels, 3, padding=1)
```


Stable Diffusion

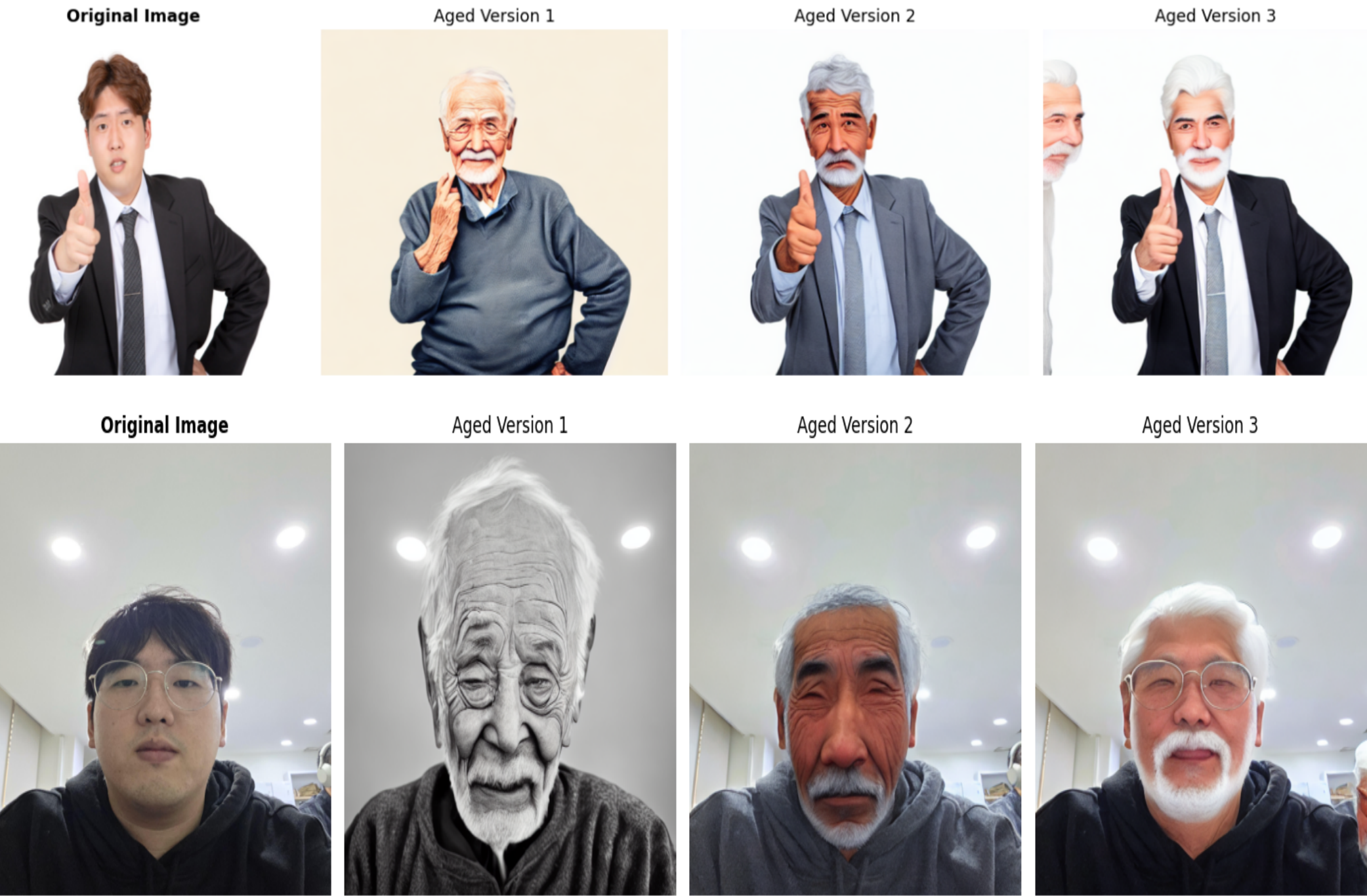
Model - InstructPix2Pix
Scheduler - DDIM Scheduler

Code

- 1) Pre-Trained Model Load
- 2) Comparison

```
def setup_model():  
    """  
    사전 학습된 InstructPix2Pix 모델 로드 (DDIM 스케줄러 사용)  
    """  
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  
    print(f'Using device: {device}')  
  
    model_id = "timbrooks/instruct-pix2pix"  
  
    print("모델 로딩 중... (처음 실행 시 다운로드에 시간이 걸릴 수 있습니다)")  
  
    pipe = StableDiffusionInstructPix2PixPipeline.from_pretrained(  
        model_id,  
        torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,  
        safety_checker=None,  
        use_safetensors=True  
    )  
  
    pipe = pipe.to(device)  
  
    # DDIM 스케줄러  
    pipe.scheduler = DDIMScheduler.from_config(pipe.scheduler.config)  
  
    print("모델 로딩 완료! (DDIM 스케줄러 사용)")  
    return pipe, device
```

2) Comparison



Stable Diffusion

Model - InstructPix2Pix

Scheduler - DDIM Scheduler

Code

1) Comparison

1) Comparison

**Original 2
(test4.jpg)**



**Aged Version 2
(70-80 years old)**



References

- Blei, David M., et al. 2017. "Variational Inference: A Review for Statisticians." *Journal of the American Statistical Association* 112(518): 859–877.
- Creswell, Antonia, et al. 2018. "Generative Adversarial Networks: An Overview." *IEEE Signal Processing Magazine* 35(1): 53–65.
- Dhariwal, Prafulla, et al. 2021. "Diffusion Models Beat GANs on Image Synthesis." arXiv 2105.05233.
- Ho, Jonathan, et al. 2020. "Denoising Diffusion Probabilistic Models." arXiv 2006.11239.
- Karras, Tero, et al. 2022. "Elucidating the Design Space of Diffusion-Based Generative Models." In *Advances in Neural Information Processing Systems* 35 (NeurIPS 2022).
- Kingma, Diederik P., and Max Welling. 2013. "Auto-Encoding Variational Bayes." arXiv 1312.6114 (ICLR 2014).
- Lu, Cheng, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. 2022. "DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps." In *Advances in Neural Information Processing Systems* 35 (NeurIPS 2022).
- Sohl-Dickstein, Jascha, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. 2015. "Deep Unsupervised Learning Using Nonequilibrium Thermodynamics." In *Proceedings of the 32nd International Conference on Machine Learning (ICML 2015)*. arXiv 1503.03585.
- Song, Jiaming, Chenlin Meng, and Stefano Ermon. 2020. "Denoising Diffusion Implicit Models." arXiv 2010.02502 (ICLR 2021).
- Song, Yang, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. 2021. "Score-Based Generative Modeling through Stochastic Differential Equations." In *International Conference on Learning Representations (ICLR 2021)*.
- Dosovitskiy, Alexey, et al. 2021. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." *International Conference on Learning Representations (ICLR) 2021*.
- Rombach, Robin, et al. 2022. "High-Resolution Image Synthesis with Latent Diffusion Models." In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10684–10695.
- Podell, Dustin, et al. 2023. "SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis." arXiv 2307.01952.

감사합니다!

QnA

DATE

2025.10.14

DEPARTMENT

삼석사와아이들

NAME

김재현 / 김현진 / 어지유 / 이문용 / 박재익